

Xây dựng mô hình dự đoán bệnh gan trên dữ liệu ILPD: Ứng dụng Bootstrap và K-fold Cross-Validation

Nguyen Ho Bao Thien

July 30, 2026

Mục lục

1	Đặt vấn đề	3
2	Cơ sở lý thuyết	4
2.1	Phương pháp Bootstrap	4
2.2	Phương pháp K-fold Cross-Validation	6
2.3	So sánh và vai trò bổ sung của hai phương pháp	7
3	Dữ liệu và tiền xử lý	8
3.1	Nguồn dữ liệu và cấu trúc	8
3.2	Thống kê mô tả và giá trị khuyết	9
4	Phân tích khám phá dữ liệu (EDA)	10
4.1	Phân bố biến mục tiêu	10
4.2	Giới tính và bệnh gan: kiểm định Chi-square	11
4.3	Tương quan giữa các biến	11
4.4	Đa cộng tuyến: hệ số VIF	12
4.5	Phân bố các biến sinh hoá theo nhóm	14
4.6	Biến đổi log1p cho các biến lệch phải	14
4.7	VIF tính lại trên thang log	16
4.8	Thống kê mô tả theo nhóm	18
4.9	Chọn biến: Mann-Whitney U và hiệu chỉnh FDR	19

5	Thiết kế quy trình mô hình hoá	20
5.1	Tập biến đưa vào mô hình	20
5.2	Chia phân tầng và gán fold	20
5.3	SMOTE: sinh mẫu tổng hợp cho lớp thiểu số	21
5.4	Hồi quy logistic có điều chuẩn ridge	21
5.5	Đóng gói quy trình: phòng rò rỉ dữ liệu	22
5.6	Bộ chỉ số đánh giá	23
6	Bootstrap: khoảng tin cậy cho Recall	24
7	Stratified K-fold Cross-Validation	25
8	Tinh chỉnh siêu tham số bằng grid search + K-fold	27
8.1	Quy tắc phá thế hoà trong phạm vi một sai số chuẩn	28
9	Chọn ngưỡng phân loại	29
10	Đánh giá cuối cùng trên tập kiểm tra	31
10.1	Khoảng tin cậy bootstrap cho toàn bộ chỉ số	32
11	Kết luận	33
11.1	Hạn chế	34
11.2	Hướng mở rộng	34
12	Tài liệu tham khảo	35

1 Đặt vấn đề

Bệnh gan thường tiến triển âm thầm: tổn thương tích lũy trong nhiều năm trước khi xuất hiện triệu chứng rõ rệt, và tới lúc đó khả năng can thiệp đã hẹp đi đáng kể. Vì vậy việc sàng lọc sớm dựa trên các chỉ số sinh hoá thường quy — bilirubin, các men gan, albumin — có giá trị thực tiễn lớn. Bộ dữ liệu *Indian Liver Patient Dataset* (ILPD) ghi lại đúng những chỉ số này cùng với chẩn đoán cuối cùng của 583 bệnh nhân, và bài toán đặt ra là một bài toán phân loại nhị phân quen thuộc: từ hồ sơ xét nghiệm, dự đoán bệnh nhân có mắc bệnh gan hay không.

Tuy nhiên, nếu chỉ dừng ở việc khớp một mô hình rồi báo cáo một con số độ chính xác thì bài toán chưa được giải quyết trọn vẹn. Con số đó được tính trên một tập dữ liệu hữu hạn; rút một mẫu bệnh nhân khác thì nó sẽ khác đi. Hai câu hỏi thực sự cần trả lời là:

1. **Trong các cấu hình mô hình có thể xây dựng, cấu hình nào dự đoán tốt nhất trên dữ liệu chưa từng thấy?** Trả lời câu này đòi hỏi một cơ chế đánh giá khách quan, không cho mô hình “nhìn trộm” dữ liệu kiểm tra trong quá trình được chọn — đó là vai trò của K-fold Cross-Validation.
2. **Mô hình cuối cùng đáng tin cậy tới mức nào?** Hiệu năng báo cáo là một điểm ước lượng; cần biết nó dao động trong khoảng nào nếu ta gặp một nhóm bệnh nhân khác — đó là vai trò của Bootstrap.

Nói gọn: **K-fold Cross-Validation giúp CHỌN mô hình một cách đúng đắn, còn Bootstrap giúp TRẢ LỜI mô hình đó có đáng tin hay không.** Tiểu luận này trình bày cơ sở lý thuyết của hai phương pháp, rồi áp dụng chúng vào một quy trình hoàn chỉnh trên ILPD, từ tiền xử lý và chọn biến cho tới tinh chỉnh siêu tham số, chọn ngưỡng phân loại và ước lượng khoảng tin cậy cho hiệu năng cuối cùng.

Một điểm cần nói rõ về cách cài đặt: toàn bộ phần thực hành chỉ dùng **base R** và gói **stats**. Các thuật toán SMOTE, VIF, chia fold phân tầng, vòng lặp bootstrap và hồi quy logistic có điều chuẩn ridge đều được viết lại từ đầu thay vì gọi caret, glmnet hay DMwR. Lựa chọn này ban đầu xuất phát từ một ràng buộc kỹ thuật của môi trường máy tính sử dụng, nhưng giữ lại được vì nó phục vụ đúng mục tiêu của tiểu luận: khi mỗi bước resampling được viết tường minh, cơ chế bên trong của nó — vốn là chủ đề chính ở đây — trở nên rõ ràng hơn nhiều so với việc gọi một hàm đóng gói sẵn.

2 Cơ sở lý thuyết

Hai phương pháp được trình bày trong chương này — Bootstrap và K-fold Cross-Validation — đều thuộc nhóm kỹ thuật *resampling* (lấy mẫu lại) và cùng xuất phát từ một khó khăn nền tảng của thống kê ứng dụng: nhà nghiên cứu chỉ quan sát được *một* mẫu hữu hạn rút ra từ tổng thể, trong khi mọi đại lượng tính từ mẫu đó — giá trị trung bình, hệ số hồi quy, hay độ chính xác của một mô hình — đều là biến ngẫu nhiên và sẽ nhận giá trị khác đi nếu ta rút được một mẫu khác. Câu hỏi cốt lõi vì thế không dừng ở “giá trị ước lượng bằng bao nhiêu” mà là “ước lượng đó dao động đến mức nào và có thể tin cậy tới đâu”. Khi công thức lý thuyết cho phân phối lấy mẫu không tồn tại hoặc quá phức tạp, các phương pháp *resampling* cho phép *mô phỏng* lại chính sự biến thiên đó từ dữ liệu sẵn có, bằng cách sinh ra nhiều mẫu con và quan sát đại lượng quan tâm thay đổi ra sao qua các mẫu con ấy.

2.1 Phương pháp Bootstrap

Bootstrap là phương pháp *resampling* do Bradley Efron giới thiệu năm 1979, dùng để ước lượng độ chính xác — sai số chuẩn, độ chệch, khoảng tin cậy — của một đại lượng thống kê mà không cần giả định về dạng phân phối của tổng thể và không cần thu thập thêm dữ liệu.

Nền tảng lý thuyết. Giả sử ta quan tâm tới một tham số tổng thể $\theta = \theta(F)$, trong đó F là hàm phân phối chưa biết của tổng thể; từ mẫu quan sát $x_1, \dots, x_n$ ta tính được ước lượng $\hat{\theta} = \theta(\hat{F})$. Để đánh giá độ chính xác của $\hat{\theta}$, về mặt lý tưởng ta cần biết *phân phối lấy mẫu* (sampling distribution) của nó — tức tập hợp các giá trị mà $\hat{\theta}$ có thể nhận nếu quá trình rút mẫu kích thước n từ F được lặp lại vô số lần. Trong thực tế điều này bất khả thi, bởi F không biết và ta chỉ có duy nhất một mẫu. Ý tưởng then chốt của Efron là thay tổng thể chưa biết F bằng *phân phối thực nghiệm* $\hat{F}$ — phân phối rời rạc gán xác suất $1/n$ cho mỗi quan sát đã có. Theo định lý Glivenko–Cantelli, $\hat{F}$ hội tụ đều về F khi cỡ mẫu tăng, nên với n đủ lớn $\hat{F}$ là một thay thế hợp lý cho F . Việc rút một mẫu kích thước n từ $\hat{F}$ chính là lấy mẫu ngẫu nhiên **có hoàn lại** từ dữ liệu gốc. Đây là nội dung của *nguyên lý plug-in*: phân phối lấy mẫu của $\hat{\theta}$ dưới F được xấp xỉ bằng phân phối của các ước lượng bootstrap $\hat{\theta}^*$ tính dưới $\hat{F}$.

Thuật toán.

1. Từ mẫu gốc kích thước n , lấy mẫu ngẫu nhiên **có hoàn lại** kích thước n , gọi là mẫu bootstrap thứ b .
2. Tính thống kê quan tâm $\hat{\theta}_b^*$ (trung bình, trung vị, hệ số hồi quy, recall, ...) trên mẫu bootstrap này.

3. Lặp lại bước 1–2 B lần (B thường từ 500 đến vài nghìn), thu được $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$.
4. Dùng phân phối thực nghiệm của B giá trị này để ước lượng sai số chuẩn và khoảng tin cậy của $\hat{\theta}$.

Ước lượng sai số chuẩn và khoảng tin cậy. Sai số chuẩn bootstrap được ước lượng bằng độ lệch chuẩn của B giá trị thống kê:

$$\widehat{SE}_{boot} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(\hat{\theta}_b^* - \bar{\theta}^* \right)^2}, \quad \bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b^*.$$

Khoảng tin cậy theo *phương pháp phân vị* (percentile) ở mức tin cậy $1 - \alpha$ là $[\hat{\theta}_{(\alpha/2)}^*, \hat{\theta}_{(1-\alpha/2)}^*]$, với $\hat{\theta}_{(q)}^*$ là phân vị mức q của tập B giá trị bootstrap; chẳng hạn ở mức 95% ta lấy phân vị 2,5% và 97,5%. Một cách tinh vi hơn là *BCa* (*bias-corrected and accelerated*), hiệu chỉnh thêm độ chệch và độ xiên của phân phối bootstrap để cho khoảng tin cậy chính xác hơn khi phân phối không đối xứng. Trong tiêu luận này phương pháp phân vị được sử dụng, vì phân phối bootstrap thu được (Hình 5) khá cân đối và không nằm sát biên $[0, 1]$ — hai tình huống mà BCa mới thực sự cần thiết.

Một biến thể phổ biến: .632 bootstrap. Trong mỗi mẫu bootstrap kích thước n , xác suất một quan sát cụ thể không được chọn ở một lần rút là $1 - 1/n$; sau n lần rút, xác suất nó hoàn toàn vắng mặt là $(1 - 1/n)^n \rightarrow e^{-1} \approx 0,368$. Do đó trung bình mỗi mẫu bootstrap chỉ chứa khoảng 63,2% số quan sát khác nhau, còn 36,8% quan sát còn lại (*out-of-bag*, OOB) không tham gia huấn luyện và có thể dùng làm tập kiểm tra độc lập. Biến thể **.632 bootstrap** khai thác tính chất này để ước lượng sai số dự đoán của mô hình, bằng cách kết hợp sai số huấn luyện (thường quá lạc quan) với sai số trên phần out-of-bag theo trọng số cố định:

$$\widehat{Err}_{.632} = 0,368 \cdot \overline{err} + 0,632 \cdot \widehat{Err}_{OOB},$$

trong đó $\overline{err}$ là sai số trên tập huấn luyện và $\widehat{Err}_{OOB}$ là sai số trung bình trên các quan sát OOB; trọng số 0,632 phản ánh đúng tỉ lệ quan sát trung bình có mặt trong mẫu bootstrap. Phiên bản cải tiến .632+ còn điều chỉnh trọng số này theo mức độ overfitting của mô hình. Biến thể này *không* được dùng ở phần thực hành, vì vai trò ước lượng sai số dự đoán ở đây đã do K-fold CV đảm nhiệm; bootstrap được dành riêng cho nhiệm vụ mà nó mạnh hơn — dựng khoảng tin cậy cho các chỉ số đo trên tập kiểm tra độc lập.

Ưu điểm. Không đòi hỏi giả định về phân phối tổng thể; áp dụng được cho hầu như mọi thống kê, kể cả những thống kê không có công thức sai số chuẩn lý thuyết (trung vị, hệ số trong mô hình phi tuyến, recall, macro F1, ...).

Nhược điểm. Tốn chi phí tính toán do phải lặp lại hàng trăm đến hàng nghìn lần; có thể cho kết quả chệch với cỡ mẫu nhỏ; đồng thời giả định ngầm rằng dữ liệu độc lập và cùng phân phối (i.i.d.), nên không áp dụng trực tiếp được cho dữ liệu có cấu trúc phụ thuộc như chuỗi thời gian (khi đó cần dùng biến thể *block bootstrap*).

2.2 Phương pháp K-fold Cross-Validation

K-fold Cross-Validation (kiểm định chéo k phần) là kỹ thuật resampling **không hoàn lại**, dùng để ước lượng sai số dự đoán của một mô hình trên dữ liệu chưa từng thấy (generalization error), qua đó tránh việc đánh giá mô hình ngay trên chính dữ liệu đã dùng để huấn luyện nó.

Nền tảng lý thuyết. Khi xây dựng một mô hình dự đoán, mục tiêu không phải là mô tả thật khớp dữ liệu đã có mà là dự đoán chính xác trên dữ liệu *mới*. Nếu đánh giá mô hình ngay trên dữ liệu huấn luyện, ta sẽ thu được ước lượng quá lạc quan, bởi mô hình đã “học thuộc” cả những dao động ngẫu nhiên riêng của mẫu — hiện tượng *overfitting*. Cách khắc phục đơn giản nhất là tách riêng một phần dữ liệu làm tập kiểm tra (holdout), nhưng cách này vừa lãng phí dữ liệu, vừa cho kết quả phụ thuộc mạnh vào một lần chia ngẫu nhiên duy nhất, khiến ước lượng có phương sai lớn khi mẫu nhỏ. K-fold Cross-Validation khắc phục đồng thời cả hai hạn chế bằng cách luân phiên vai trò huấn luyện/kiểm tra qua k lần chia, sao cho mọi quan sát đều được dùng để kiểm tra đúng một lần và dùng để huấn luyện $k - 1$ lần.

Thuật toán.

1. Chia ngẫu nhiên dữ liệu thành k phần (fold) rời nhau, kích thước xấp xỉ bằng nhau.
2. Với mỗi $i = 1, \dots, k$: dùng $k - 1$ phần còn lại để huấn luyện mô hình, phần thứ i dùng để kiểm tra.
3. Tính sai số dự đoán trên từng fold, sau đó lấy trung bình:

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k L\left(y_i, \hat{f}^{(-i)}(x_i)\right),$$

với L là hàm mất mát (tỉ lệ phân loại sai, MSE, ...) và $\hat{f}^{(-i)}$ là mô hình được huấn luyện mà không dùng fold thứ i .

Ví dụ minh họa. Xét bài toán phân loại bệnh gan trên ILPD với $n = 583$ quan sát và mô hình hồi quy logistic. Với $k = 5$, dữ liệu được chia thành 5 phần, mỗi phần khoảng 117 quan sát. Ở lần lặp thứ nhất, 4 phần (khoảng 466 quan

sát) được dùng để ước lượng các hệ số của mô hình, phần còn lại dùng để dự đoán và tính sai số phân loại; quá trình lặp lại 5 lần, mỗi phần lần lượt đóng vai tập kiểm tra. Trung bình 5 sai số này là ước lượng sai số dự đoán của mô hình, còn độ lệch chuẩn giữa chúng phản ánh mức độ ổn định của mô hình qua các cách chia dữ liệu khác nhau.

Một biến thể phổ biến: Stratified k-fold. Khi biến mục tiêu mất cân bằng lớp, việc chia fold hoàn toàn ngẫu nhiên có thể tạo ra những fold có tỉ lệ lớp lệch hẳn so với tổng thể, thậm chí thiếu vắng lớp thiểu số, làm cho ước lượng sai số bị nhiễu. **Stratified k-fold** (kiểm định chéo phân tầng) khắc phục bằng cách chia sao cho *mỗi fold giữ nguyên tỉ lệ các lớp* như trong toàn bộ dữ liệu. Với ILPD — nơi tỉ lệ giữa nhóm mắc bệnh và không mắc bệnh xấp xỉ 71%/29% — mỗi fold được ràng buộc để duy trì đúng tỉ lệ đó, nhờ vậy mỗi lần kiểm tra đều phản ánh đúng cơ cấu lớp thực tế và cho ước lượng ổn định hơn. Đây là biến thể được áp dụng trong toàn bộ phần thực hành. Ngoài ra còn hai biến thể đáng chú ý khác: *Leave-One-Out CV* (trường hợp đặc biệt khi $k = n$) và *Repeated k-fold* (lặp lại toàn bộ quy trình nhiều lần với các cách chia khác nhau để giảm thêm phương sai của ước lượng).

Vai trò trong quy trình Data Science. K-fold CV không chỉ dùng để *đánh giá* một mô hình đơn lẻ, mà còn là công cụ khách quan để *so sánh nhiều cấu hình* với nhau và *tinh chỉnh siêu tham số* (hyperparameter tuning). Điều kiện tiên quyết là mọi bước có học tham số từ dữ liệu phải được thực hiện hoàn toàn bên trong vòng lặp CV, nhằm tránh hiện tượng “nhìn trộm” tập kiểm tra (data leakage) khi lựa chọn mô hình — một yêu cầu được thảo luận kỹ ở Mục 5.5.

Ưu điểm. Mọi quan sát đều được dùng để kiểm tra đúng một lần nên ước lượng sai số ít lạc quan hơn so với chỉ chia train/test một lần; áp dụng được cho bất kỳ thuật toán học máy nào.

Nhược điểm. Chi phí tính toán tăng tuyến tính theo k (hoặc theo n với LOOCV); kết quả có thể có phương sai cao khi k nhỏ, vì mỗi lần chỉ ước lượng trên một phần nhỏ dữ liệu.

2.3 So sánh và vai trò bổ sung của hai phương pháp

Hai phương pháp có chung bản chất resampling nhưng phục vụ hai mục tiêu khác nhau và đánh đổi bias–variance theo hai hướng ngược nhau:

	Bootstrap	K-fold CV
Cách lấy mẫu	Có hoàn lại	Không hoàn lại
Mục tiêu chính	Ước lượng độ tin cậy (SE, CI) của một thống kê	Ước lượng sai số dự đoán, chọn/só sánh mô hình
Xu hướng bias	Có thể lạc quan nếu không hiệu chỉnh (.632+)	Xấp xỉ không chệch với sai số dự đoán thật
Xu hướng variance	Thấp hơn (nhiều mẫu, chồng lấp nhiều)	Cao hơn khi k nhỏ (mỗi fold độc lập hơn)
Chi phí tính toán	Cao (B thường 500–10000 lần lặp)	Vừa phải (k thường 5–10 lần lặp)

Nói cách khác: **K-fold CV giúp CHỌN mô hình đúng đắn** (dựa trên ước lượng khách quan về khả năng tổng quát hoá), còn **Bootstrap giúp TRẢ LỜI mô hình đã chọn đó có đáng tin cậy hay không** (thông qua khoảng tin cậy cho hiệu năng). Đây chính là trình tự được áp dụng trong phần thực hành: K-fold CV chạy trên tập huấn luyện để tinh chỉnh siêu tham số và chọn ngưỡng phân loại, trong khi tập kiểm tra được niêm phong tới bước cuối và chỉ được dùng một lần duy nhất, với Bootstrap dựng khoảng tin cậy quanh các chỉ số đo trên đó.

3 Dữ liệu và tiền xử lý

3.1 Nguồn dữ liệu và cấu trúc

ILPD được thu thập tại vùng đông bắc bang Andhra Pradesh, Ấn Độ, gồm 583 hồ sơ bệnh nhân với 10 biến giải thích: tuổi, giới tính và tám chỉ số sinh hoá. Tập CSV gốc không có dòng tiêu đề nên tên cột được gán thủ công. Nhãn gốc mã hoá 1 = có bệnh và 2 = không bệnh; ta đổi về quy ước 1/0 quen thuộc để các công thức đếm TP/FP/TN/FN ở phần sau viết được gọn.

```
col_names <- c(
  "Age", "Gender", "Total_Bilirubin", "Direct_Bilirubin",
  "Alkaline_Phosphotase", "Alamine_Aminotransferase",
  "Aspartate_Aminotransferase", "Total_Protiens", "Albumin",
  "Albumin_and_Globulin_Ratio", "target"
)

df <- read.csv("Indian Liver Patient Dataset (ILPD).csv",
  header = FALSE, col.names = col_names,
  stringsAsFactors = FALSE)

# Nhãn gốc: 1 = có bệnh, 2 = không bệnh. Đổi về 1 / 0 cho tiện tính toán.
df$target <- ifelse(df$target == 2, 0, 1)

cat("Kích thước dữ liệu:", nrow(df), "dòng x", ncol(df), "cột\n")

## Kích thước dữ liệu: 583 dòng x 11 cột

cat("Phân bố target:", sum(df$target == 1), "có bệnh /",
  sum(df$target == 0), "không bệnh\n")

## Phân bố target: 416 có bệnh / 167 không bệnh
```



```
str(df)

## 'data.frame': 583 obs. of 11 variables:
## $ Age : int 65 62 62 58 72 46 26 29 17 55 ...
## $ Gender : chr "Female" "Male" "Male" "Male" ...
## $ Total_Bilirubin : num 0.7 10.9 7.3 1 3.9 1.8 0.9 0.9 0.9 0.7 ...
## $ Direct_Bilirubin : num 0.1 5.5 4.1 0.4 2 0.7 0.2 0.3 0.3 0.2 ...
## $ Alkaline_Phosphotase : int 187 699 490 182 195 208 154 202 202 290 ...
## $ Alamine_Aminotransferase : int 16 64 60 14 27 19 16 14 22 53 ...
## $ Aspartate_Aminotransferase: int 18 100 68 20 59 14 12 11 19 58 ...
## $ Total_Protiens : num 6.8 7.5 7 6.8 7.3 7.6 7 6.7 7.4 6.8 ...
## $ Albumin : num 3.3 3.2 3.3 3.4 2.4 4.4 3.5 3.6 4.1 3.4 ...
## $ Albumin_and_Globulin_Ratio: num 0.9 0.74 0.89 1 0.4 1.3 1 1.1 1.2 1 ...
## $ target : num 1 1 1 1 1 1 1 0 1 ...
```

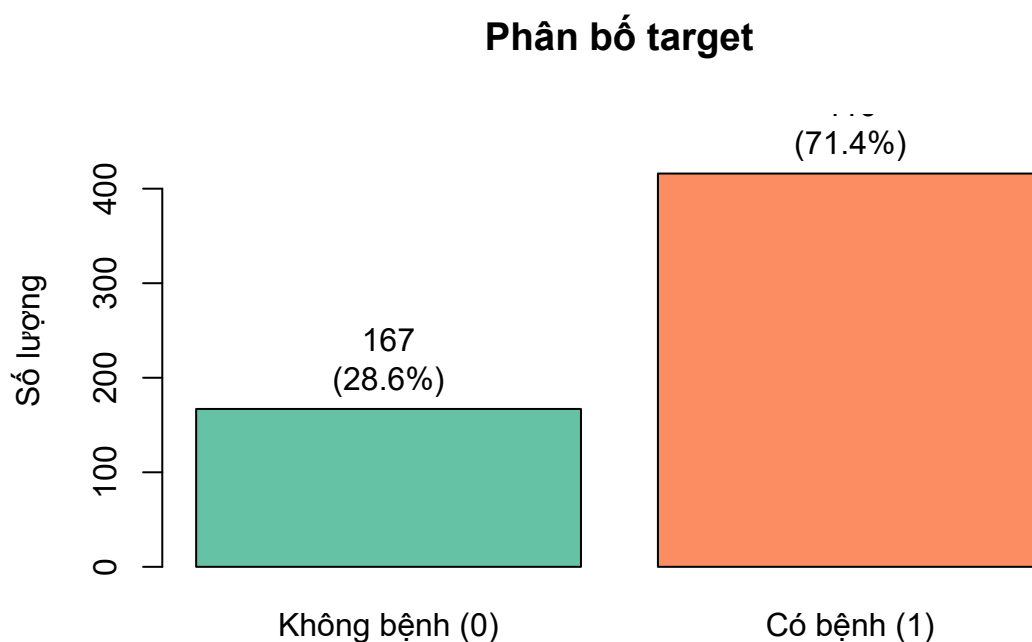
3.2 Thống kê mô tả và giá trị khuyết

```
summary(df)

##      Age      Gender  Total_Bilirubin  Direct_Bilirubin
## Min.   : 4.00   Length :583   Min.    : 0.400   Min.    : 0.100
## 1st Qu.:33.00   N.unique : 2   1st Qu.: 0.800   1st Qu.: 0.200
## Median :45.00   N.blank  : 0   Median : 1.000   Median : 0.300
## Mean   :44.75   Min.nchar: 4   Mean   : 3.299   Mean    : 1.486
## 3rd Qu.:58.00   Max.nchar: 6   3rd Qu.: 2.600   3rd Qu.: 1.300
## Max.    :90.00           Max.    :75.000   Max.    :19.700
##
## Alkaline_Phosphotase Alamine_Aminotransferase
## Min.    : 63.0      Min.    : 10.00
## 1st Qu.: 175.5      1st Qu.: 23.00
## Median : 208.0      Median : 35.00
## Mean    : 290.6      Mean    : 80.71
## 3rd Qu.: 298.0      3rd Qu.: 60.50
## Max.    :2110.0     Max.    :2000.00
##
## Aspartate_Aminotransferase Total_Protiens      Albumin
## Min.    : 10.0      Min.    :2.700   Min.    :0.900
## 1st Qu.: 25.0      1st Qu.:5.800   1st Qu.:2.600
## Median : 42.0      Median :6.600   Median :3.100
## Mean    :109.9      Mean    :6.483   Mean    :3.142
## 3rd Qu.: 87.0      3rd Qu.:7.200   3rd Qu.:3.800
## Max.    :4929.0     Max.    :9.600   Max.    :5.500
##
## Albumin_and_Globulin_Ratio      target
## Min.    :0.3000      Min.    :0.0000
## 1st Qu.:0.7000      1st Qu.:0.0000
## Median :0.9300      Median :1.0000
## Mean    :0.9471      Mean    :0.7136
## 3rd Qu.:1.1000      3rd Qu.:1.0000
## Max.    :2.8000      Max.    :1.0000
## NAs      :4

colSums(is.na(df))

##      Age      Gender
##      0      0
## Total_Bilirubin  Direct_Bilirubin
##      0      0
## Alkaline_Phosphotase Alamine_Aminotransferase
##      0      0
## Aspartate_Aminotransferase Total_Protiens
##      0      0
## Albumin Albumin_and_Globulin_Ratio
##      0      4
## target
##      0
```



Hình 1: Phân bố biến mục tiêu: dữ liệu mất cân bằng lớp theo tỉ lệ xấp xỉ 71/29

Chỉ có 4 giá trị khuyết, toàn bộ nằm ở cột `Albumin_and_Globulin_Ratio`. Con số này rất nhỏ so với 583 quan sát nên cách xử lý gần như không ảnh hưởng tới kết quả; tuy vậy ta vẫn điền khuyết bằng trung vị chứ không xoá dòng, và — điểm quan trọng hơn nhiều — việc điền khuyết được đặt *bên trong* quy trình huấn luyện để trung vị chỉ được tính từ tập huấn luyện. Lý do được trình bày ở Mục 5.5.

4 Phân tích khám phá dữ liệu (EDA)

4.1 Phân bố biến mục tiêu

```
counts <- table(df$target)
pct <- round(100 * counts / sum(counts), 1)

bp <- barplot(counts,
  names.arg = c("Không bệnh (0)", "Có bệnh (1)"),
  col = c("#66c2a5", "#fc8d62"),
  main = "Phân bố target", ylab = "Số lượng",
  ylim = c(0, max(counts) * 1.15))
text(bp, counts, labels = paste0(counts, "\n(", pct, "%)"), pos = 3)
```

Tỉ lệ 71,4% / 28,6% cho thấy dữ liệu mất cân bằng ở mức trung bình. Hệ quả trực tiếp: *độ chính xác tổng thể* (accuracy) là chỉ số gây hiểu nhầm — một mô

hình dự đoán tất cả bệnh nhân đều mắc bệnh đã đạt accuracy 0,714 mà không học được gì. Vì vậy toàn bộ phần đánh giá về sau sẽ báo cáo recall và precision của *từng lớp* cùng với macro F1, chứ không dựa vào accuracy.

4.2 Giới tính và bệnh gan: kiểm định Chi-square

Trước khi đưa Gender vào mô hình, cần kiểm tra xem biến này có thực sự liên hệ với biến mục tiêu hay không. Giả thuyết H_0 : tỉ lệ mắc bệnh không khác nhau giữa hai giới.

```
contingency <- table(df$Gender, df$target)
print(contingency)

##
##           0    1
## Female  50   92
## Male   117  324

chi_res <- chisq.test(contingency)
cat("\nChi-square =", round(chi_res$statistic, 4),
    "| p-value =", round(chi_res$p.value, 4), "\n")

##
## Chi-square = 3.5466 | p-value = 0.0597

cat("Bảng tần số kỳ vọng (cần >= 5 để kiểm định hợp lệ):\n")

## Bảng tần số kỳ vọng (cần >= 5 để kiểm định hợp lệ):

print(round(chi_res$expected, 2))

##
##           0    1
## Female  40.68 101.32
## Male   126.32 314.68
```

Tần số kỳ vọng ở cả bốn ô đều lớn hơn 5 nên kiểm định hợp lệ. Với $p = 0,0597 > 0,05$, ta *không* đủ bằng chứng bác bỏ H_0 : dữ liệu chưa cho thấy khác biệt có ý nghĩa thống kê về tỉ lệ mắc bệnh giữa nam và nữ. Cần lưu ý cách diễn đạt — “không bác bỏ được H_0 ” khác với “chứng minh H_0 đúng”; giá trị p nằm sát ngưỡng 0,05 nên kết luận hợp lý là bằng chứng còn yếu chứ không phải liên hệ chắc chắn không tồn tại.

4.3 Tương quan giữa các biến

```
numeric_cols <- c(
  "Age", "Total_Bilirubin", "Direct_Bilirubin", "Alkaline_Phosphotase",
  "Alamine_Aminotransferase", "Aspartate_Aminotransferase",
  "Total_Protiens", "Albumin", "Albumin_and_Globulin_Ratio"
)

corr_mat <- cor(df[, c(numeric_cols, "target")], use = "complete.obs")

# Rút gọn tên chỉ để in cho vừa khổ giấy; corr_mat gốc giữ nguyên
```

```
corr_disp <- round(corr_mat, 2)
dimnames(corr_disp) <- list(abbreviate(rownames(corr_mat), 12),
                           abbreviate(colnames(corr_mat), 6))
print(corr_disp)
```

```
##           Age Ttl_Bl Drct_B Alkl_P Almn_A Aspr_A Ttl_Pr Albumn
## Age      1.00  0.01  0.01  0.08 -0.09 -0.02 -0.19 -0.26
## Total_Bilrbn 0.01  1.00  0.87  0.21  0.21  0.24 -0.01 -0.22
## Direct_Blrbn 0.01  0.87  1.00  0.23  0.23  0.26  0.00 -0.23
## Alkl_Phsph 0.08  0.21  0.23  1.00  0.12  0.17 -0.03 -0.16
## Almn_Amntns -0.09  0.21  0.23  0.12  1.00  0.79 -0.04 -0.03
## Asprtt_Amnt -0.02  0.24  0.26  0.17  0.79  1.00 -0.03 -0.08
## Total_Prots -0.19 -0.01  0.00 -0.03 -0.04 -0.03  1.00  0.78
## Albumin    -0.26 -0.22 -0.23 -0.16 -0.03 -0.08  0.78  1.00
## Albmn_nd_G_R -0.22 -0.21 -0.20 -0.23  0.00 -0.07  0.23  0.69
## target     0.13  0.22  0.25  0.18  0.16  0.15 -0.03 -0.16
##           A__R target
## Age      -0.22  0.13
## Total_Bilrbn -0.21  0.22
## Direct_Blrbn -0.20  0.25
## Alkl_Phsph -0.23  0.18
## Almn_Amntns  0.00  0.16
## Asprtt_Amnt -0.07  0.15
## Total_Prots  0.23 -0.03
## Albumin     0.69 -0.16
## Albmn_nd_G_R 1.00 -0.16
## target     -0.16  1.00
```

```
par(mar = c(9, 9, 3, 2))
image(seq_len(ncol(corr_mat)), seq_len(nrow(corr_mat)),
      t(corr_mat[nrow(corr_mat):1, ]),
      col = colorRampPalette(c("#3288bd", "white", "#d53e4f"))(64),
      zlim = c(-1, 1), axes = FALSE, xlab = "", ylab = "",
      main = "Ma trận tương quan")
axis(1, seq_len(ncol(corr_mat)), colnames(corr_mat), las = 2, cex.axis = 0.7)
axis(2, seq_len(nrow(corr_mat)), rev(rownames(corr_mat)), las = 2,
      cex.axis = 0.7)
```

```
par(mar = c(5, 4, 4, 2) + 0.1)
```

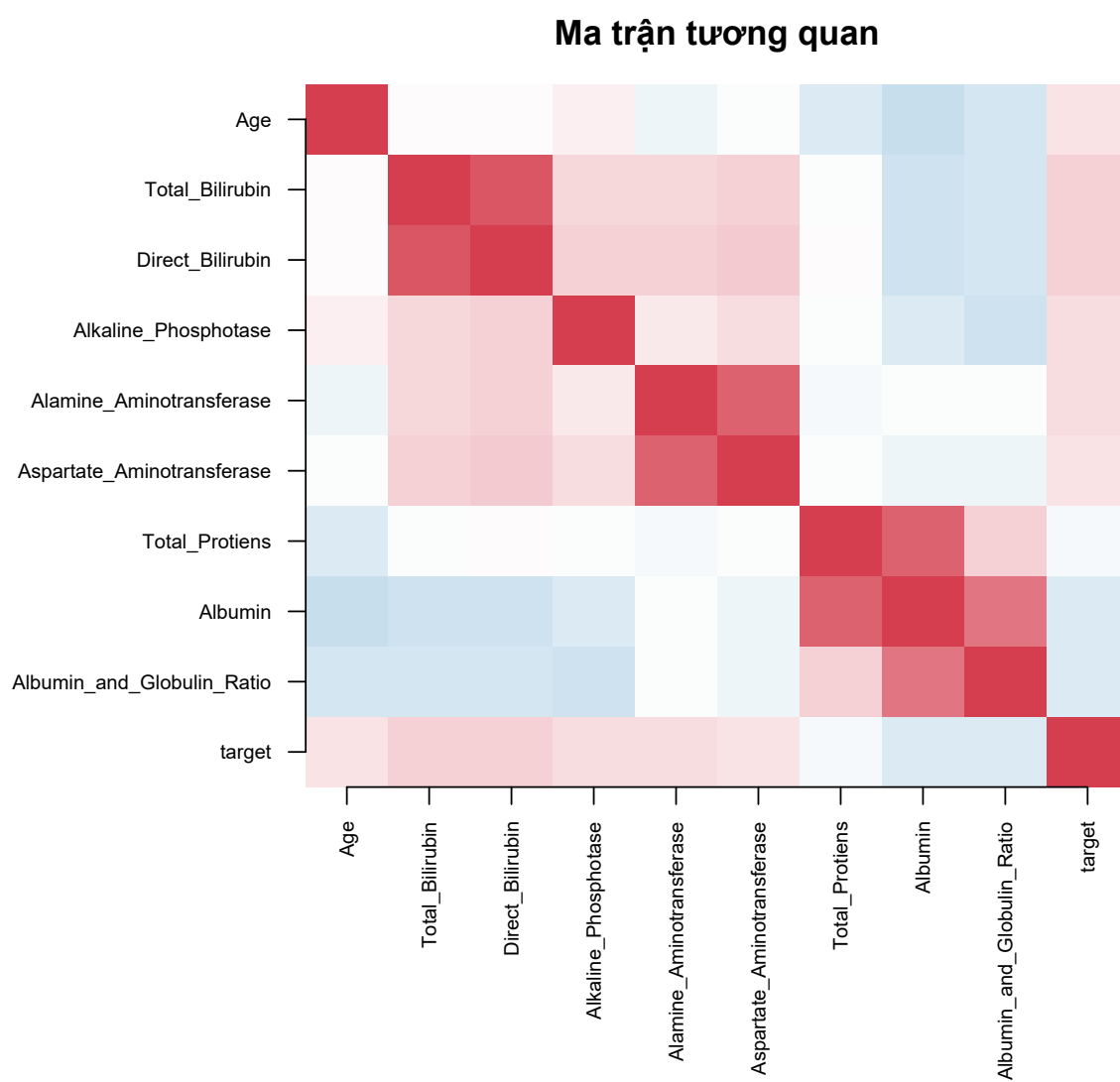
Ba cặp biến có tương quan rất cao nổi lên rõ rệt: Total_Bilirubin với Direct_Bilirubin ($r = 0,87$), Alamine_Aminotransferase với Aspartate_Aminotransferase ($r = 0,79$), và Total_Protiens với Albumin ($r = 0,78$). Điều này dễ hiểu về mặt sinh hoá — bilirubin trực tiếp là một thành phần của bilirubin toàn phần, còn albumin là thành phần chính của protein toàn phần — nhưng lại là vấn đề đối với hồi quy logistic, vì các biến gần như trùng thông tin làm ma trận thiết kế gần suy biến và hệ số ước lượng mất ổn định. Đồng thời, tương quan với target của mọi biến đều yếu (giá trị lớn nhất là 0,25), báo trước rằng không có biến đơn lẻ nào tách được hai nhóm một cách rõ ràng.

4.4 Đa cộng tuyến: hệ số VIF

Tương quan từng cặp chỉ phát hiện được quan hệ hai biến. Hệ số phóng đại phương sai (Variance Inflation Factor) đo mức độ một biến bị giải thích bởi *tất cả* các biến còn lại:

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

với R_j^2 là hệ số xác định của hồi quy biến j lên tất cả các biến khác. Hàm dưới đây tự cài đặt công thức đó bằng `lm()`, không cần gói car.



Hình 2: Ma trận tương quan giữa các biến sinh hoá và biến mục tiêu

```
compute_vif <- function(data) {
  vars <- colnames(data)
  sapply(vars, function(v) {
    others <- setdiff(vars, v)
    fml <- as.formula(paste(v, "~", paste(others, collapse = " + ")))
    r2 <- summary(lm(fml, data = data))$r.squared
    1 / (1 - r2)
  })
}

X_vif <- df[, c(numeric_cols, "Gender")]
X_vif$Gender <- ifelse(X_vif$Gender == "Male", 1, 0)
X_vif$Albumin_and_Globulin_Ratio[is.na(X_vif$Albumin_and_Globulin_Ratio)] <-
  median(X_vif$Albumin_and_Globulin_Ratio, na.rm = TRUE)

vif_raw <- compute_vif(X_vif)
print(round(sort(vif_raw, decreasing = TRUE), 3))

##           Albumin           Total_Protiens
##           10.124           5.549
##      Direct_Bilirubin      Total_Bilirubin
##           4.525           4.277
## Albumin_and_Globulin_Ratio      Alamine_Aminotransferase
##           3.683           2.820
## Aspartate_Aminotransferase      Alkaline_Phosphotase
##           2.811           1.122
##           Age           Gender
##           1.099           1.034
```

Theo quy ước thường dùng, $VIF > 5$ là đáng lo và $VIF > 10$ là nghiêm trọng. Albumin vượt ngưỡng 10 và Total_Protiens vượt ngưỡng 5, đúng như cặp tương quan 0,78 đã gợi ý.

4.5 Phân bố các biến sinh hoá theo nhóm

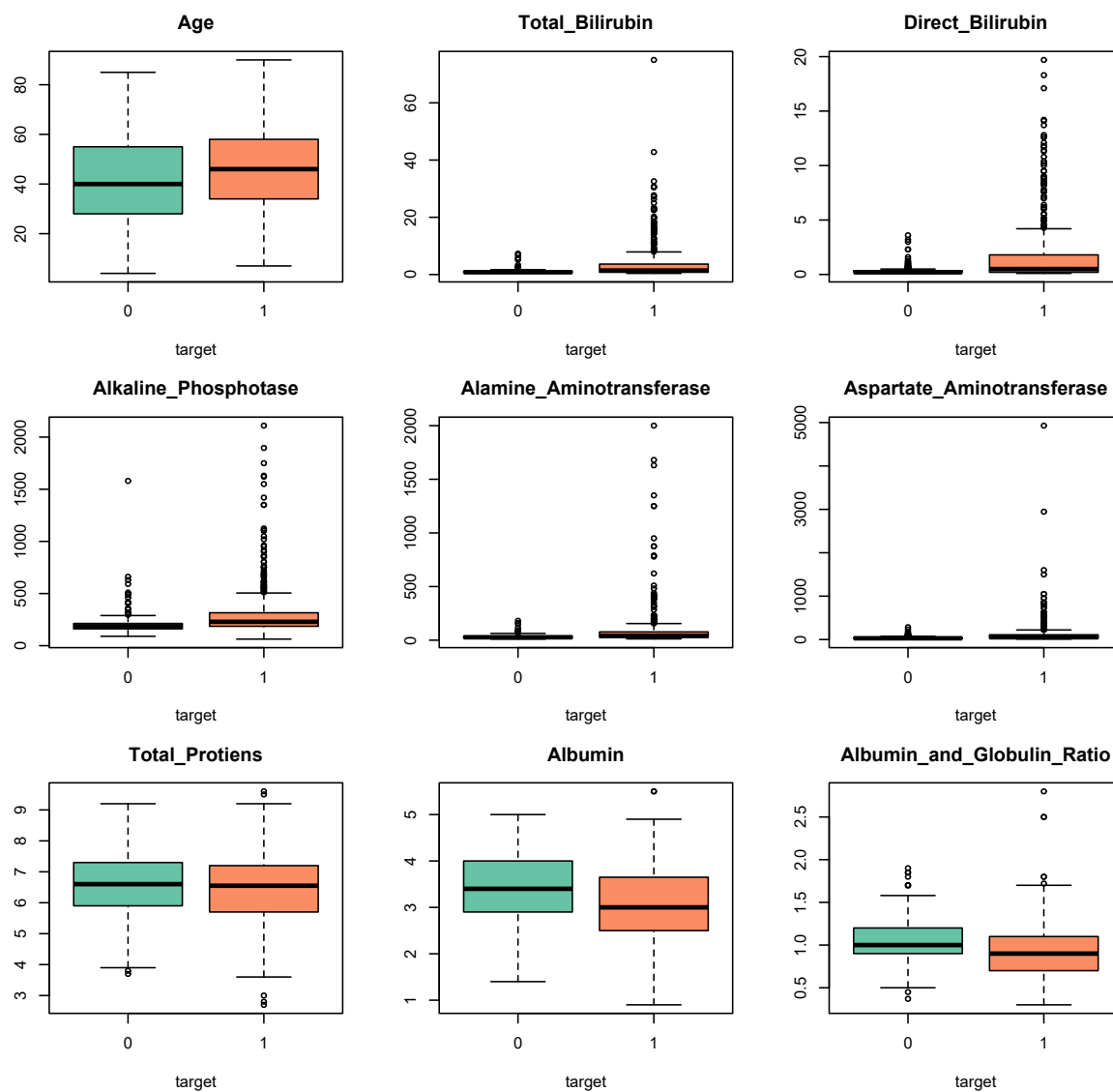
```
par(mfrow = c(3, 3), mar = c(4, 4, 3, 1))
for (col in numeric_cols) {
  boxplot(df[[col]] ~ df$target, col = c("#66c2a5", "#fc8d62"),
    main = col, xlab = "target", ylab = "")
}
```

```
par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)
```

Hình 3 cho thấy năm biến — hai loại bilirubin và ba men gan — lệch phải cực nặng: hộp bị nén sát trục còn các điểm ngoại lai kéo dài lên trên. Đây là dạng phân bố điển hình của chỉ số men gan.

4.6 Biến đổi log1p cho các biến lệch phải

Cách xử lý ngoại lai quen thuộc là winsorization (kẹp giá trị ở phân vị 1% và 99%). Ở đây cách đó *không* phù hợp: các giá trị cực đại là tín hiệu lâm sàng thật — AST khoảng 5000 U/L tương ứng với suy gan cấp — và chúng tập trung ở nhóm bệnh, nên kẹp chúng lại chính là xóa đi thông tin phân biệt hai lớp. Phép biến đổi $\log(1 + x)$ là lựa chọn thích hợp hơn: nó nén đuôi phân phối mà vẫn *bảo toàn thứ hạng* của các quan sát, do là hàm đơn điệu tăng.



Hình 3: Phân bố 9 biến số theo nhóm bệnh, thang đo gốc. Các men gan lệch phải rất nặng

```

skewed_cols <- c("Total_Bilirubin", "Direct_Bilirubin",
                "Alkaline_Phosphotase", "Alamine_Aminotransferase",
                "Aspartate_Aminotransferase")

df_raw <- df # giữ bản gốc để tra thang đo lâm sàng
for (col in skewed_cols) df[[col]] <- log1p(df[[col]])

# Hệ số bất đối xứng, tự tính vì base R không có hàm skewness
skewness <- function(x) {
  x <- x[!is.na(x)]
  mean((x - mean(x))^3) / (mean((x - mean(x))^2)^1.5)
}

print(round(data.frame(
  goc = sapply(df_raw[skewed_cols], skewness),
  sau_log1p = sapply(df[skewed_cols], skewness)
), 2))

##               goc sau_log1p
## Total_Bilirubin    4.89    1.72
## Direct_Bilirubin   3.20    1.68
## Alkaline_Phosphotase 3.76    1.33
## Alamine_Aminotransferase 6.53    1.47
## Aspartate_Aminotransferase 10.52    1.23

```

Hiệu quả rất rõ: Aspartate_Aminotransferase giảm hệ số bất đối xứng từ 10,52 xuống 1,23, Alamine_Aminotransferase từ 6,53 xuống 1,47. Cả năm biến đều về mức bất đối xứng nhẹ.

```

par(mfrow = c(3, 3), mar = c(4, 4, 3, 1))
for (col in numeric_cols) {
  boxplot(df[[col]] ~ df$target, col = c("#66c2a5", "#fc8d62"),
    main = if (col %in% skewed_cols) paste0("log1p(", col, ")") else col,
    xlab = "target", ylab = "")
}

```

```

par(mfrow = c(1, 1), mar = c(5, 4, 4, 2) + 0.1)

```

4.7 VIF tính lại trên thang log

Một điểm dễ bị bỏ qua: hệ số tương quan Pearson và VIF *không* bất biến với phép biến đổi log (khác với các kiểm định dựa trên thứ hạng). Ma trận thiết kế mà mô hình thực sự sử dụng là ma trận trên thang log, nên VIF phải được tính lại trên đúng thang đó.

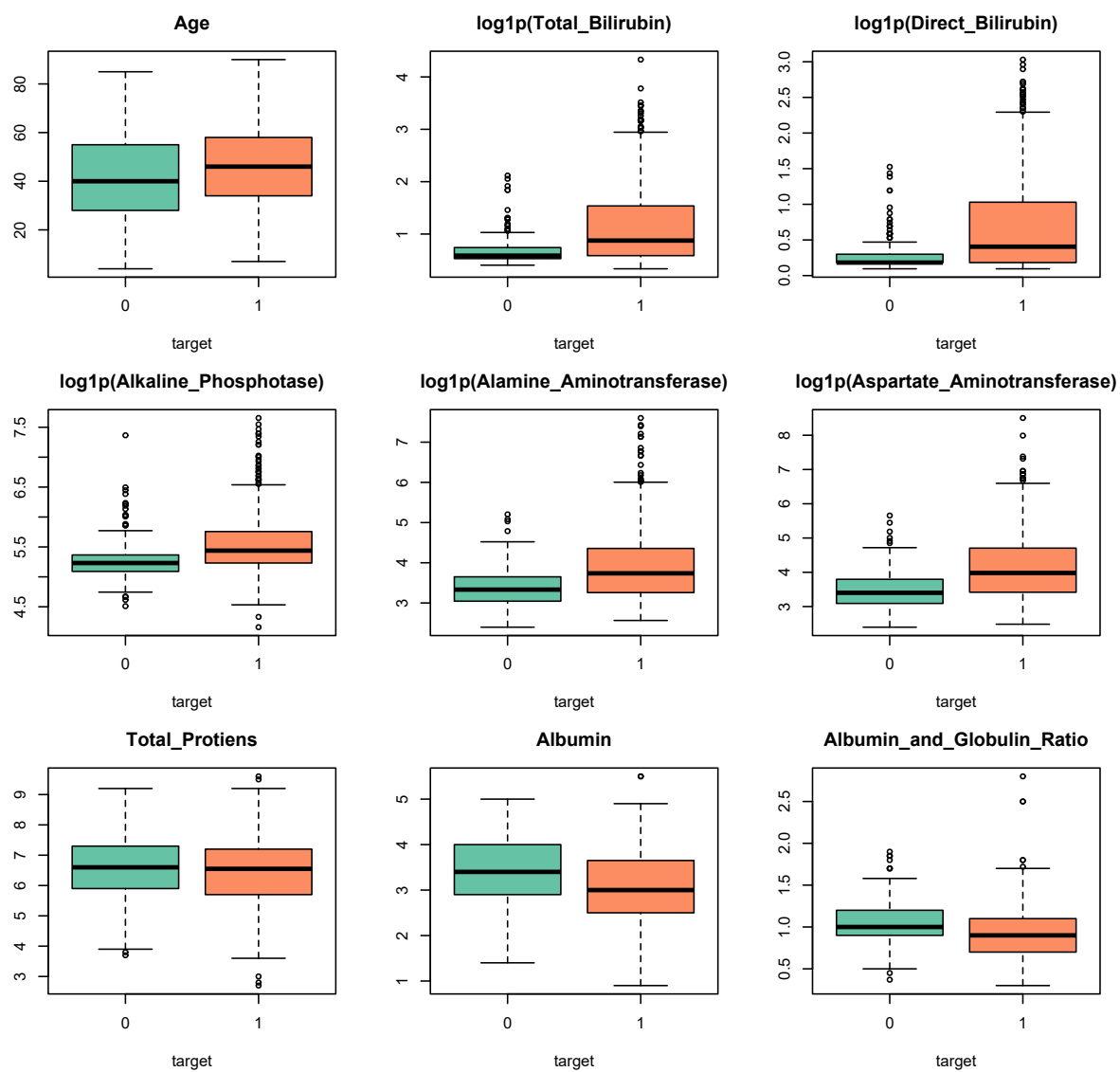
```

X_vif_log <- X_vif
for (col in skewed_cols) X_vif_log[[col]] <- log1p(df_raw[[col]])
vif_log <- compute_vif(X_vif_log)

print(round(data.frame(
  VIF_goc = vif_raw,
  VIF_log = vif_log[names(vif_raw)],
  chenh_lech = vif_log[names(vif_raw)] - vif_raw
)[order(-vif_log[names(vif_raw)]), ], 2))

##               VIF_goc VIF_log chenh_lech
## Total_Bilirubin    4.28   20.72    16.44
## Direct_Bilirubin    4.53   20.55    16.03

```

Hình 4: Phân bố sau khi biến đổi log1p. Khác biệt giữa hai nhóm trở nên nhìn được

## Albumin	10.12	10.62	0.49
## Total_Protiens	5.55	5.65	0.10
## Aspartate_Aminotransferase	2.81	4.18	1.37
## Alamine_Aminotransferase	2.82	3.94	1.12
## Albumin_and_Globulin_Ratio	3.68	3.75	0.07
## Alkaline_Phosphotase	1.12	1.28	0.16
## Age	1.10	1.10	0.00
## Gender	1.03	1.06	0.03

Kết quả thay đổi hẳn bức tranh. Trên thang gốc, hai biến bilirubin có VIF chỉ khoảng 4,3–4,5; trên thang log chúng vọt lên trên 20 — mức nghiêm trọng. Nguyên nhân là quan hệ giữa hai biến này vốn có dạng nhân tính, và phép lấy log đã biến nó thành quan hệ tuyến tính gần như hoàn hảo. Nếu chỉ nhìn VIF trên thang gốc, ta sẽ kết luận sai rằng cặp bilirubin không có vấn đề. Đây là căn cứ trực tiếp để loại Direct_Bilirubin khỏi mô hình.

4.8 Thống kê mô tả theo nhóm

Bảng dưới đây được tính trên `df_raw` — thang đo gốc — để các con số còn giữ nguyên đơn vị lâm sàng và có thể đối chiếu với khoảng tham chiếu xét nghiệm.

```
for (col in numeric_cols) {
  s <- tapply(df_raw[[col]], df_raw$target, function(x) {
    x <- x[!is.na(x)]
    c(min = min(x), max = max(x), mean = mean(x),
      median = median(x), sd = sd(x))
  })
  cat("\n", col, "\n")
  print(round(do.call(rbind, s), 2))
}
```

```
##
## Age
##   min max  mean median  sd
## 0   4  85 41.24    40 17.00
## 1   7  90 46.15    46 15.65
##
## Total_Bilirubin
##   min max mean median  sd
## 0 0.5  7.3 1.14    0.8 1.00
## 1 0.4 75.0 4.16    1.4 7.14
##
## Direct_Bilirubin
##   min max mean median  sd
## 0 0.1  3.6 0.40    0.2 0.52
## 1 0.1 19.7 1.92    0.5 3.21
##
## Alkaline_Phosphotase
##   min max  mean median  sd
## 0  90 1500 219.75    186 140.99
## 1  63 2110 319.01    229 268.31
##
## Alamine_Aminotransferase
##   min max  mean median  sd
## 0  10 181 33.65    27 25.06
## 1  12 2000 99.61    41 212.77
##
## Aspartate_Aminotransferase
##   min max  mean median  sd
## 0  10 285 40.69    29.0 36.41
## 1  11 4929 137.70    52.5 337.39
##
## Total_Protiens
##   min max mean median  sd
## 0 3.7 9.2 6.54    6.60 1.06
## 1 2.7 9.6 6.46    6.55 1.09
##
## Albumin
##   min max mean median  sd
## 0 1.4 5.0 3.34    3.4 0.78
## 1 0.9 5.5 3.06    3.0 0.79
##
## Albumin_and_Globulin_Ratio
##   min max mean median  sd
## 0 0.37 1.9 1.03    1.0 0.29
## 1 0.30 2.8 0.91    0.9 0.33
```

Chênh lệch giữa trung bình và trung vị ở nhóm bệnh rất lớn — chẳng hạn Aspartate_Aminotransferase có trung bình 137,7 nhưng trung vị chỉ 52,5 — một lần nữa khẳng định phân phối lệch nặng và củng cố lựa chọn dùng kiểm định phi tham số ở bước tiếp theo.

4.9 Chọn biến: Mann-Whitney U và hiệu chỉnh FDR

Vì phần lớn biến lệch phải nặng, kiểm định t (giả định phân phối chuẩn) không phù hợp. Ta dùng kiểm định Mann-Whitney U, dựa trên thứ hạng nên không đòi hỏi giả định về dạng phân phối — và cũng vì thế, giá trị p của nó *không đổi* sau phép biến đổi $\log_{10}$ ở trên.

Kích thước ảnh hưởng đi kèm là hệ số tương quan *rank-biserial*, nằm trong $[-1, 1]$:

$$r_{rb} = 1 - \frac{2U}{n_0 n_1}.$$

Riêng biến Gender là biến phân loại nên dùng kết quả Chi-square ở trên với kích thước ảnh hưởng Cramér's V.

Một vấn đề nữa cần xử lý: ta chạy 10 kiểm định cùng lúc ở mức $\alpha = 0,05$. Xác suất có ít nhất một kết quả dương tính giả khi đó không còn là 5% nữa. Thủ tục Benjamini–Hochberg hiệu chỉnh giá trị p để kiểm soát *tỉ lệ phát hiện sai* (False Discovery Rate).

```
fs <- data.frame(feature = character(), p_value = numeric(),
                 effect_size = numeric(), stringsAsFactors = FALSE)

for (col in numeric_cols) {
  g0 <- df[[col]][df$target == 0]
  g1 <- df[[col]][df$target == 1]
  w <- wilcox.test(g0, g1, exact = FALSE)
  n0 <- sum(!is.na(g0)); n1 <- sum(!is.na(g1))
  rb <- 1 - 2 * w$statistic / (n0 * n1) # rank-biserial correlation
  fs <- rbind(fs, data.frame(feature = col, p_value = w$p.value,
                             effect_size = as.numeric(rb)))
}

# Gender: dùng Cramer's V làm kích thước ảnh hưởng
n_total <- sum(contingency)
cramers_v <- sqrt(chi_res$statistic /
                  (n_total * (min(dim(contingency)) - 1)))
fs <- rbind(fs, data.frame(feature = "Gender", p_value = chi_res$p.value,
                           effect_size = as.numeric(cramers_v)))

fs$p_fdr <- p.adjust(fs$p_value, method = "BH") # Benjamini-Hochberg
fs$significant <- fs$p_fdr < 0.05
fs <- fs[order(fs$p_value), ]

fs_disp <- fs
fs_disp$p_value <- signif(fs_disp$p_value, 3)
fs_disp$effect_size <- round(fs_disp$effect_size, 3)
fs_disp$p_fdr <- signif(fs_disp$p_fdr, 3)
print(fs_disp, row.names = FALSE)

##           feature p_value effect_size p_fdr significant
## Aspartate_Aminotransferase 9.21e-14      0.394 9.21e-13      TRUE
##      Total_Bilirubin 2.29e-13      0.387 1.14e-12      TRUE
```

##	Direct_Bilirubin	7.43e-13	0.372	2.48e-12	TRUE
##	Alamine_Aminotransferase	2.33e-12	0.371	5.83e-12	TRUE
##	Alkaline_Phosphotase	4.35e-11	0.349	8.69e-11	TRUE
##	Albumin_and_Globulin_Ratio	5.81e-06	-0.240	9.69e-06	TRUE
##	Albumin	5.57e-05	-0.213	7.95e-05	TRUE
##	Age	1.77e-03	0.165	2.22e-03	TRUE
##	Gender	5.97e-02	0.078	6.63e-02	FALSE
##	Total_Protiens	4.37e-01	-0.041	4.37e-01	FALSE

Tám trên mười biến có ý nghĩa thống kê sau hiệu chỉnh FDR. Hai biến bị loại là Gender ($p_{FDR} = 0,066$) và Total_Protiens ($p_{FDR} = 0,437$). Nhóm men gan và bilirubin có kích thước ảnh hưởng lớn nhất (khoảng 0,35–0,39), trong khi Age tuy có ý nghĩa thống kê nhưng kích thước ảnh hưởng chỉ 0,17 — một lời nhắc rằng ý nghĩa thống kê và ý nghĩa thực tiễn là hai chuyện khác nhau.

5 Thiết kế quy trình mô hình hoá

5.1 Tập biến đưa vào mô hình

Từ 10 biến ban đầu, ba biến bị loại với lý do khác nhau: Direct_Bilirubin vì trùng thông tin với Total_Bilirubin ($r = 0,87$, VIF trên thang log khoảng 20); Gender và Total_Protiens vì không đạt ngưỡng FDR.

```
model_cols <- c("Age", "Total_Bilirubin", "Alkaline_Phosphotase",
               "Alamine_Aminotransferase", "Aspartate_Aminotransferase",
               "Albumin", "Albumin_and_Globulin_Ratio")

X_all <- as.matrix(df[, model_cols])
y_all <- df$target
```

Cần thừa nhận rằng đa cộng tuyến chưa được loại bỏ hoàn toàn: Albumin và Albumin_and_Globulin_Ratio vẫn liên quan chặt về mặt định nghĩa, và cặp men gan ALT/AST vẫn còn tương quan 0,79. Đây là một lý do nữa để dùng hồi quy có điều chuẩn ridge ở Mục 5.4 — ridge xử lý được đa cộng tuyến còn lại bằng cách co các hệ số tương quan về phía nhau thay vì để chúng dao động ngược chiều.

5.2 Chia phân tầng và gán fold

Cả hai hàm dưới đây đều xáo trộn *trong nội bộ từng lớp* rồi mới chia, nhờ vậy tỉ lệ 71/29 được giữ nguyên ở mọi tập con.

```
stratified_split <- function(y, test_frac = 0.2, seed = 42) {
  set.seed(seed)
  test_idx <- integer(0)
  for (cls in sort(unique(y))) {
    idx <- which(y == cls)
    test_idx <- c(test_idx, sample(idx, round(length(idx) * test_frac)))
  }
  list(train = setdiff(seq_along(y), test_idx), test = sort(test_idx))
}
```

```

}

stratified_folds <- function(y, k = 5, seed = 42) {
  set.seed(seed)
  fold <- integer(length(y))
  for (cls in sort(unique(y))) {
    idx <- sample(which(y == cls))
    fold[idx] <- rep_len(seq_len(k), length(idx))
  }
  fold
}

```

5.3 SMOTE: sinh mẫu tổng hợp cho lớp thiểu số

Với tỉ lệ lớp 71/29, mô hình huấn luyện trực tiếp sẽ thiên vị lớp đa số. SMOTE (Synthetic Minority Over-sampling Technique) cân bằng lại bằng cách sinh thêm mẫu *tổng hợp* cho lớp thiểu số, thay vì nhân bản mẫu có sẵn — nhân bản chỉ làm tăng trọng số của đúng những điểm đã có và dễ dẫn tới overfitting. Mỗi mẫu mới được nội suy tuyến tính giữa một quan sát thiểu số x_a và một trong k láng giềng gần nhất cùng lớp của nó:

$$x_{new} = x_a + u \cdot (x_b - x_a), \quad u \sim \mathcal{U}(0, 1),$$

tức là một điểm ngẫu nhiên nằm trên đoạn thẳng nối hai quan sát thật.

```

smote_oversample <- function(X, y, k = 5, seed = 42) {
  set.seed(seed)
  tab <- table(y)
  min_lab <- as.numeric(names(tab)[which.min(tab)])
  n_new <- as.integer(max(tab) - min(tab))
  if (n_new <= 0) return(list(X = X, y = y))

  X_min <- X[y == min_lab, , drop = FALSE]
  if (nrow(X_min) <= k) k <- max(1, nrow(X_min) - 1)

  # Khoảng cách trong nội bộ Lớp thiểu số; chặn đường chéo để không tự chọn mình
  D <- as.matrix(dist(X_min))
  diag(D) <- Inf

  synth <- matrix(0, nrow = n_new, ncol = ncol(X))
  for (i in seq_len(n_new)) {
    a <- sample.int(nrow(X_min), 1)
    neighbours <- order(D[a, ])[seq_len(k)] # k láng giềng gần nhất
    b <- neighbours[sample.int(k, 1)]        # chọn ngẫu nhiên 1 trong k
    gap <- runif(1)                          # điểm mới nằm trên đoạn a-b
    synth[i, ] <- X_min[a, ] + gap * (X_min[b, ] - X_min[a, ])
  }
  colnames(synth) <- colnames(X)
  list(X = rbind(X, synth), y = c(y, rep(min_lab, n_new)))
}

```

5.4 Hồi quy logistic có điều chuẩn ridge

Hàm `glm()` của base R không hỗ trợ phạt L_2 , còn `glmnet` là gói biên dịch nằm ngoài phạm vi cài đặt đã chọn. Ta tự cài thuật toán IRLS (Iteratively Reweighted

Least Squares). Bài toán cần giải là cực đại hoá log-likelihood có phạt:

$$\ell_{\lambda}(\beta) = \sum_{i=1}^n \left[y_i \eta_i - \log(1 + e^{\eta_i}) \right] - \frac{\lambda}{2} \sum_{j=1}^p \beta_j^2, \quad \eta_i = x_i^{\top} \beta,$$

trong đó hệ số chặn β_0 *không* bị phạt. Mỗi vòng lặp IRLS quy bài toán về một bài toán bình phương tối thiểu có trọng số:

$$\beta^{(t+1)} = (X^{\top} W X + P)^{-1} X^{\top} W z,$$

với $\mu_i = \sigma(\eta_i)$, trọng số $W = \text{diag}(\mu_i(1 - \mu_i))$, biến phản hồi hiệu chỉnh $z = \eta + W^{-1}(y - \mu)$, và $P = \lambda I$ với phần tử P_{11} đặt bằng 0.

```
fit_logreg_ride <- function(X, y, lambda = 0, max_iter = 50, tol = 1e-8) {
  Xd <- cbind(1, X) # thêm cột hệ số chặn
  p_dim <- ncol(Xd)
  beta <- rep(0, p_dim)

  P <- diag(lambda, p_dim)
  P[1, 1] <- 0 # không điều chuẩn hệ số chặn

  for (it in seq_len(max_iter)) {
    eta <- as.vector(Xd %*% beta)
    mu <- 1 / (1 + exp(-eta)) # xác suất dự đoán
    W <- mu * (1 - mu) # trọng số IRLS
    W[W < 1e-10] <- 1e-10 # chặn dưới để ma trận không suy biến
    z <- eta + (y - mu) / W # biến phản hồi hiệu chỉnh

    beta_new <- tryCatch(
      solve(t(Xd) %*% (Xd * W) + P, t(Xd) %*% (W * z)),
      error = function(e) beta
    )
    if (max(abs(beta_new - beta)) < tol) { beta <- beta_new; break }
    beta <- as.vector(beta_new)
  }
  as.vector(beta)
}

predict_proba <- function(beta, X) {
  1 / (1 + exp(-as.vector(cbind(1, X) %*% beta)))
}
```

5.5 Đóng gói quy trình: phòng rò rỉ dữ liệu

Đây là mục quan trọng nhất về mặt phương pháp luận. Quy trình huấn luyện gồm bốn bước, trong đó *ba bước đầu đều học tham số từ dữ liệu*:

1. điền khuyết — học trung vị của từng cột;
2. chuẩn hoá — học trung bình và độ lệch chuẩn của từng cột;
3. SMOTE — sinh mẫu mới dựa trên cấu trúc láng giềng của dữ liệu;
4. khớp mô hình ridge.

Nếu ba bước đầu được chạy trên *toàn bộ* dữ liệu trước khi chia fold — cách làm rất phổ biến vì tiện — thì thông tin từ tập kiểm tra đã rò rỉ vào quá trình huấn luyện: trung vị và độ lệch chuẩn dùng để biến đổi tập huấn luyện đã được tính có sự đóng góp của chính những quan sát mà sau đó ta dùng để đánh giá. Kết quả báo cáo khi ấy lạc quan giả tạo. Riêng với SMOTE, hậu quả còn nặng hơn nhiều: nếu chạy trước khi chia, các mẫu tổng hợp nội suy từ quan sát nằm trong tập kiểm tra sẽ lọt vào tập huấn luyện, khiến mô hình gần như được xem trước đáp án.

Cách phòng duy nhất đúng là gói cả bốn bước vào một hàm và *gọi hàm đó bên trong mỗi fold*, để nó chỉ nhìn thấy phần dữ liệu huấn luyện của fold ấy.

```
fit_pipeline <- function(X_tr, y_tr, lambda = 1, smote_k = 5, seed = 42) {
  # 1. Điền khuyết bằng median CỦA TẬP TRAIN
  med <- apply(X_tr, 2, median, na.rm = TRUE)
  for (j in seq_len(ncol(X_tr))) X_tr[is.na(X_tr[, j]), j] <- med[j]

  # 2. Chuẩn hoá theo mean/sd CỦA TẬP TRAIN
  mu <- colMeans(X_tr)
  sdv <- apply(X_tr, 2, sd)
  sdv[sdv == 0] <- 1
  X_tr_s <- scale(X_tr, center = mu, scale = sdv)

  # 3. SMOTE chỉ trên tập train
  res <- smote_oversample(X_tr_s, y_tr, k = smote_k, seed = seed)

  # 4. Khớp mô hình
  beta <- fit_logreg_ridge(res$X, res$y, lambda = lambda)

  list(beta = beta, median = med, mean = mu, sd = sdv)
}

# Áp dụng chuỗi biến đổi đã học từ train lên dữ liệu mới
pipeline_proba <- function(fit, X_new) {
  for (j in seq_len(ncol(X_new))) X_new[is.na(X_new[, j]), j] <- fit$median[j]
  X_s <- scale(X_new, center = fit$mean, scale = fit$sd)
  predict_proba(fit$beta, X_s)
}
```

Lưu ý rằng `fit_pipeline` trả về cả `median`, `mean`, `sd` chứ không chỉ hệ số β : đó chính là các tham số đã học, và `pipeline_proba` phải áp lại đúng chúng lên dữ liệu mới. SMOTE không xuất hiện ở hàm dự đoán — nó chỉ là kỹ thuật can thiệp vào tập huấn luyện, không bao giờ áp lên dữ liệu cần dự đoán.

5.6 Bộ chỉ số đánh giá

```
compute_metrics <- function(y_true, y_pred) {
  tp <- sum(y_true == 1 & y_pred == 1)
  tn <- sum(y_true == 0 & y_pred == 0)
  fp <- sum(y_true == 0 & y_pred == 1)
  fn <- sum(y_true == 1 & y_pred == 0)

  recall1 <- if ((tp + fn) > 0) tp / (tp + fn) else NA # độ nhạy
  recall0 <- if ((tn + fp) > 0) tn / (tn + fp) else NA # độ đặc hiệu
  prec1 <- if ((tp + fp) > 0) tp / (tp + fp) else 0
  prec0 <- if ((tn + fn) > 0) tn / (tn + fn) else 0
  f1_1 <- if ((prec1 + recall1) > 0) 2*prec1*recall1/(prec1+recall1) else 0
  f1_0 <- if ((prec0 + recall0) > 0) 2*prec0*recall0/(prec0+recall0) else 0
}
```

```

c(accuracy = (tp + tn) / length(y_true),
  recall1 = recall1, precision1 = prec1,
  recall0 = recall0, precision0 = prec0,
  macro_f1 = (f1_1 + f1_0) / 2)
}

print_confusion <- function(y_true, y_pred, title = "CONFUSION MATRIX") {
  tp <- sum(y_true == 1 & y_pred == 1); tn <- sum(y_true == 0 & y_pred == 0)
  fp <- sum(y_true == 0 & y_pred == 1); fn <- sum(y_true == 1 & y_pred == 0)

  cat("\n", title, "\n", sep = "")
  print(matrix(c(tn, fp, fn, tp), nrow = 2, byrow = TRUE,
    dimnames = list(`Thực tế` = c("Không bệnh", "Có bệnh"),
    `Dự đoán` = c("Không bệnh", "Có bệnh"))))
  cat(sprintf(" TN = %3d không bệnh, đoán đúng\n", tn))
  cat(sprintf(" FP = %3d không bệnh nhưng báo có bệnh\n", fp))
  cat(sprintf(" FN = %3d CÓ BỆNH nhưng bỏ sót\n", fn))
  cat(sprintf(" TP = %3d có bệnh, bắt được\n", tp))
  invisible(compute_metrics(y_true, y_pred))
}

```

Macro F1 — trung bình cộng F1 của hai lớp — được chọn làm chỉ số tổng hợp vì nó cho hai lớp trọng số bằng nhau bất kể chênh lệch cỡ, đúng với tinh thần của bài toán mất cân bằng.

6 Bootstrap: khoảng tin cậy cho Recall

Recall của lớp có bệnh chỉ là *một* con số tính từ hơn trăm bệnh nhân trong tập kiểm tra. Gặp một nhóm bệnh nhân khác thì con số đó sẽ khác. Bootstrap ước lượng chính mức dao động ấy.

Điều cần nói rõ về thiết kế: mô hình được huấn luyện **một lần** và dự đoán **một lần**; vector `pred_test` sau đó cố định suốt vòng lặp. Mỗi lần lặp chỉ rút lại *tập kiểm tra* có hoàn lại. Khoảng tin cậy thu được vì thế đo độ bất định đến từ việc lấy mẫu bệnh nhân kiểm tra, chứ không bao gồm độ bất định do thay đổi tập huấn luyện — một giới hạn được bàn thêm ở phần kết luận.

```

sp <- stratified_split(y_all, test_frac = 0.2, seed = 42)
X_train <- X_all[sp$train, , drop = FALSE]; y_train <- y_all[sp$train]
X_test <- X_all[sp$test, , drop = FALSE]; y_test <- y_all[sp$test]

cat("Train:", nrow(X_train), "mẫu | Test:", nrow(X_test), "mẫu\n")

## Train: 467 mẫu | Test: 116 mẫu

# Huấn Luyện MỘT lần, dự đoán MỘT lần
fit_80 <- fit_pipeline(X_train, y_train, lambda = 1, smote_k = 5)
proba_test <- pipeline_proba(fit_80, X_test)
pred_test <- as.integer(proba_test >= 0.5)

recall_point <- compute_metrics(y_test, pred_test)["recall1"]

```

```

CONF <- 0.90
B <- 10000
n_test <- length(y_test)

recalls <- numeric(0)

```



```

for (b in seq_len(B)) {
  # Rút CỎ HOÀN LẠI, cùng cỡ mẫu
  idx <- sample.int(n_test, n_test, replace = TRUE)
  if (sum(y_test[idx] == 1) == 0) next # không có ca bệnh -> recall vô nghĩa
  recalls <- c(recalls,
               compute_metrics(y_test[idx], pred_test[idx])["recall1"])
}

# Khoảng tin cậy percentile: cắt (1-CONF)/2 ở MỖI đuôi
tail_pct <- (1 - CONF) / 2
ci <- quantile(recalls, c(tail_pct, 1 - tail_pct))

cat(sprintf("Recall lớp có bệnh   = %.3f\n", recall_point))

## Recall lớp có bệnh   = 0.614

cat(sprintf("Khoảng tin cậy %.0f%%   = [%.3f, %.3f]\n",
            CONF * 100, ci[1], ci[2]))

## Khoảng tin cậy 90%   = [0.526, 0.700]

cat(sprintf("Bề rộng khoảng       = %.3f\n", ci[2] - ci[1]))

## Bề rộng khoảng       = 0.174

cat(sprintf("Sai số chuẩn         = %.3f\n", sd(recalls)))

## Sai số chuẩn         = 0.053

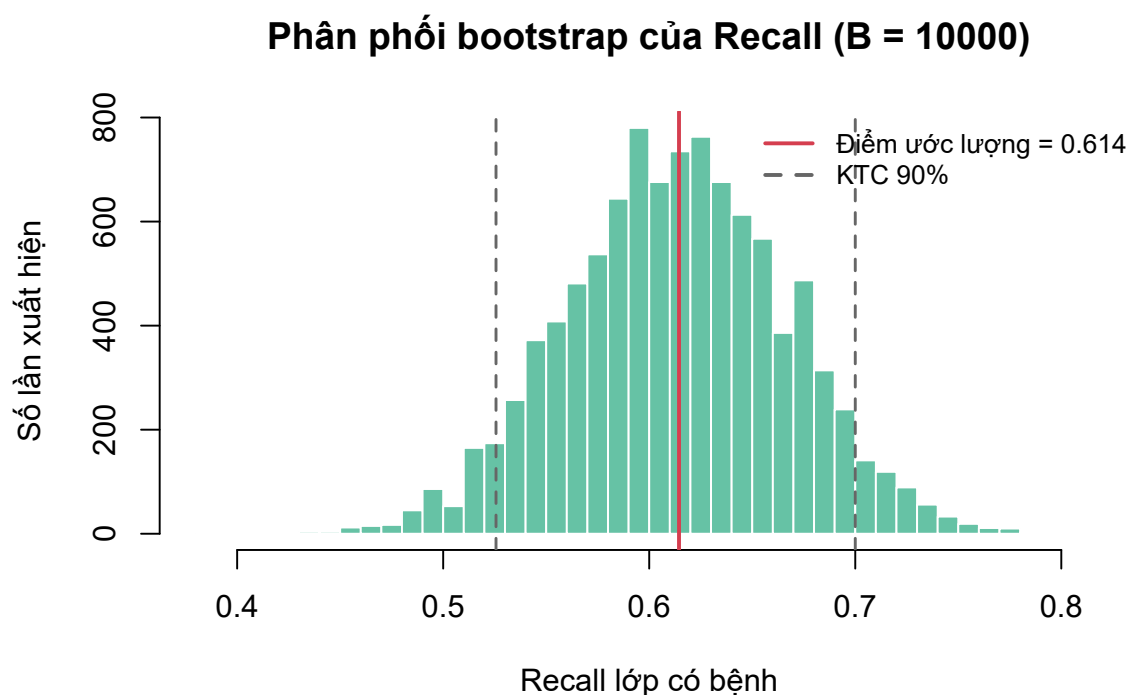
hist(recalls, breaks = 40, col = "#66c2a5", border = "white",
     main = sprintf("Phân phối bootstrap của Recall (B = %d)",
                    length(recalls)),
     xlab = "Recall lớp có bệnh", ylab = "Số lần xuất hiện")
abline(v = recall_point, col = "#d53e4f", lwd = 2)
abline(v = ci, col = "gray40", lty = 2, lwd = 1.5)
legend("topright", bty = "n", cex = 0.85,
     legend = c(sprintf("Điểm ước lượng = %.3f", recall_point),
                 sprintf("KTC %.0f%%", CONF * 100)),
     col = c("#d53e4f", "gray40"), lty = c(1, 2), lwd = 2)

```

Với ngưỡng mặc định 0,5, recall lớp có bệnh là 0,614 và khoảng tin cậy 90% là [0,526; 0,700] — bề rộng 0,174, tức gần 17 điểm phần trăm. Con số này đáng chú ý: nó có nghĩa là nếu chỉ báo cáo “recall = 0,61” thì ta đã che giấu một mức bất định rất lớn, hoàn toàn do cỡ mẫu kiểm tra chỉ có 116 quan sát, trong đó 83 ca bệnh. Đây chính là loại thông tin mà một con số điểm ước lượng đơn lẻ không bao giờ truyền tải được. Hình 5 cho thấy phân phối bootstrap tương đối cân đối, biện minh cho việc dùng khoảng tin cậy phân vị đơn giản thay vì BCa.

7 Stratified K-fold Cross-Validation

Phân trên đánh giá mô hình trên một lần chia 80/20 duy nhất. K-fold CV luân phiên vai trò của cả 5 phần, nhờ đó mọi quan sát đều được dự đoán đúng một lần bởi một mô hình chưa từng thấy nó.



Hình 5: Phân phối bootstrap của Recall lớp có bệnh với $B = 10000$

Chọn $k = 5$ có lý do cụ thể: mỗi fold kiểm tra đúng 20% dữ liệu, trùng với tỉ lệ 80:20 ở phần trước, nên hai phần cùng đánh giá mô hình được huấn luyện trên khoảng 466 mẫu và có thể so sánh trực tiếp với nhau.

```
K <- 5
folds <- stratified_folds(y_all, k = K, seed = 42)

cv_results <- data.frame()
for (i in seq_len(K)) {
  te <- which(folds == i)
  tr <- which(folds != i)

  fit_i <- fit_pipeline(X_all[tr, , drop = FALSE], y_all[tr],
                        lambda = 1, smote_k = 5)
  pred_i <- as.integer(pipeline_proba(fit_i,
                                     X_all[te, , drop = FALSE]) >= 0.5)

  m <- compute_metrics(y_all[te], pred_i)
  cv_results <- rbind(cv_results, data.frame(
    Fold = i, n_train = length(tr), n_test = length(te),
    Accuracy = m["accuracy"], Recall = m["recall1"],
    Macro_F1 = m["macro_f1"]
  ))
}
rownames(cv_results) <- NULL
print(round(cv_results, 4), row.names = FALSE)

## Fold n_train n_test Accuracy Recall Macro_F1
## 1 465 118 0.6186 0.5238 0.6124
## 2 466 117 0.6923 0.6386 0.6776
## 3 467 116 0.6466 0.6145 0.6263
## 4 467 116 0.6466 0.6265 0.6230
## 5 467 116 0.6897 0.6265 0.6758

for (m in c("Accuracy", "Recall", "Macro_F1")) {
```

```
cat(sprintf(" %-9s = %.3f +/- %.3f\n",
            m, mean(cv_results[[m]]), sd(cv_results[[m]])))
}

## Accuracy = 0.659 +/- 0.032
## Recall = 0.606 +/- 0.047
## Macro_F1 = 0.643 +/- 0.031
```

Kết quả được báo cáo dạng *trung bình $\pm$ độ lệch chuẩn giữa các fold*, không phải một con số đơn lẻ. Độ lệch chuẩn ở đây có ý nghĩa riêng: nó cho biết kết quả dao động bao nhiêu khi đổi sang một cách chia dữ liệu khác. Recall dao động mạnh nhất ($0,606 \pm 0,047$, trải từ 0,524 ở fold 1 tới 0,639 ở fold 2), trong khi macro F1 ổn định hơn ($0,643 \pm 0,031$).

Điểm đáng chú ý là recall trung bình qua 5 fold (0,606) rất gần với recall đo trên một lần chia 80/20 ở phần trước (0,614), và cả hai đều nằm gọn trong khoảng tin cậy bootstrap [0,526; 0,700]. Hai phương pháp resampling độc lập cho kết quả nhất quán — một dấu hiệu tốt cho thấy ước lượng không phải sản phẩm của một lần chia may mắn.

8 Tinh chỉnh siêu tham số bằng grid search + K-fold

Từ đây trở đi, quy trình tuân thủ nghiêm ngặt nguyên tắc phân vai: **K-fold CV chạy trên tập huấn luyện để tinh chỉnh, tập kiểm tra bị niêm phong** và chỉ được mở ở phần cuối cùng.

Hai siêu tham số được dò: cường độ điều chuẩn λ của ridge và số láng giềng k mà SMOTE dùng để nội suy. Tiêu chí chấm điểm là macro F1 chứ *không* phải recall thuần — tối đa hoá recall lớp 1 có một nghiệm tầm thường là dự đoán tất cả đều “có bệnh”, cho recall bằng 1 nhưng hoàn toàn vô dụng.

```
param_grid <- expand.grid(
  lambda = c(0.01, 0.1, 1, 10, 100), # cường độ điều chuẩn ridge
  smote_k = c(3, 5, 7)                # số láng giềng SMOTE
)

folds_inner <- stratified_folds(y_train, k = 5, seed = 42)

grid_mean <- grid_sd <- numeric(nrow(param_grid))
for (g in seq_len(nrow(param_grid))) {
  scores <- numeric(5)
  for (i in seq_len(5)) {
    te <- which(folds_inner == i); tr <- which(folds_inner != i)
    fit_g <- fit_pipeline(X_train[tr, , drop = FALSE], y_train[tr],
                        lambda = param_grid$lambda[g],
                        smote_k = param_grid$smote_k[g])
    pred_g <- as.integer(pipeline_proba(
      fit_g, X_train[te, , drop = FALSE]) >= 0.5)
    scores[i] <- compute_metrics(y_train[te], pred_g)["macro_f1"]
  }
  grid_mean[g] <- mean(scores)
  grid_sd[g] <- sd(scores) # dao động giữa 5 fold, dùng để đo mức nhiễu
}
```

```
param_grid$macro_f1 <- grid_mean
param_grid$sd_fold <- grid_sd

print(head(param_grid[order(-param_grid$macro_f1), ], 5), row.names = FALSE)

## lambda smote_k macro_f1 sd_fold
## 100      7 0.6440162 0.01687905
## 100      5 0.6408597 0.01098893
## 100      3 0.6407688 0.01654468
## 1       7 0.6406804 0.03143136
## 10      7 0.6398884 0.02943170
```

8.1 Quy tắc phá thế hoà trong phạm vi một sai số chuẩn

Lấy thẳng `which.max` là một sai lầm phổ biến khi các bộ tham số cho điểm chênh nhau ít hơn chính sai số của phép đo. Ở đây bộ đứng đầu đạt $\text{macro F1} = 0,6440$ còn bộ thứ năm đạt $0,6399$ — chênh lệch $0,004$, trong khi sai số chuẩn của trung bình 5 fold là $0,0075$. Trong tình huống đó, “bộ thắng” chỉ đơn giản là bộ gặp may với cách chia fold cụ thể này; chọn nó không có cơ sở thống kê nào.

Cách xử lý: coi mọi bộ nằm trong phạm vi *một sai số chuẩn* quanh điểm cao nhất là ngang tài nhau, rồi phá hoà bằng một tiêu chí có lý do rõ ràng.

```
i_top <- which.max(param_grid$macro_f1)
se_top <- param_grid$sd_fold[i_top] / sqrt(5) # sai số chuẩn của trung bình
tied <- which(param_grid$macro_f1 >= param_grid$macro_f1[i_top] - se_top)

cand <- tied[param_grid$lambda[tied] == min(param_grid$lambda[tied])]
best_idx <- cand[which.max(param_grid$macro_f1[cand])]
best <- param_grid[best_idx, ]

cat(sprintf("Điểm cao nhất: %.4f (lambda = %g), sai số chuẩn = %.4f\n",
  param_grid$macro_f1[i_top], param_grid$lambda[i_top], se_top))

## Điểm cao nhất: 0.6440 (lambda = 100), sai số chuẩn = 0.0075

cat(sprintf("Có %d/%d bộ nằm trong phạm vi một sai số chuẩn\n",
  length(tied), nrow(param_grid)))

## Có 14/15 bộ nằm trong phạm vi một sai số chuẩn

cat(sprintf("Phá hoà bằng lambda nhỏ nhất -> lambda = %g, smote_k = %g\n",
  best$lambda, best$smote_k))

## Phá hoà bằng lambda nhỏ nhất -> lambda = 0.01, smote_k = 3

cat(sprintf("Đã thử %d bộ x 5 fold = %d lần khớp mô hình\n",
  nrow(param_grid), nrow(param_grid) * 5))

## Đã thử 15 bộ x 5 fold = 75 lần khớp mô hình
```

Kết quả cho thấy 14 trên 15 bộ tham số nằm trong phạm vi một sai số chuẩn — nói cách khác, lưới tham số này gần như không phân biệt được các cấu hình với nhau, và bất kỳ lựa chọn nào dựa trên chênh lệch điểm số đều là lựa chọn dựa trên nhiễu.

Tiêu chí phá hoà được dùng là **chọn λ nhỏ nhất**. Cần nhấn mạnh rằng điều này *ngược* với quy tắc 1-SE quen thuộc trong tài liệu, vốn ưu tiên mô hình điều chuẩn mạnh hơn cho gọn. Lý do đảo chiều nằm ở bước tiếp theo của quy trình: ta sẽ chỉnh ngưỡng phân loại, mà việc đó cần xác suất dự đoán trải rộng. λ lớn nén các hệ số lại, kéo xác suất co về quanh 0,5, khiến ngưỡng trở nên cực kỳ nhạy — dịch một chút là recall của hai lớp đảo lộn. Khi các bộ đã ngang điểm nhau trong phạm vi nhiều, chọn bộ cho xác suất trải rộng hơn là lựa chọn có căn cứ.

9 Chọn ngưỡng phân loại

Ngưỡng mặc định 0,5 tối ưu cho độ chính xác tổng thể và không ưu tiên lớp nào. Bài toán y tế thì khác: bỏ sót một ca bệnh (FN) tốn kém hơn nhiều so với báo động nhầm một người khoẻ (FP), vì FP chỉ dẫn tới một xét nghiệm bổ sung còn FN có thể khiến bệnh tiến triển không được phát hiện. Vì vậy ta hạ ngưỡng xuống để đẩy recall lớp có bệnh lên.

Ngưỡng *phải* được dò trên tập huấn luyện. Dò trên tập kiểm tra thì tập kiểm tra đã tham gia vào một quyết định, và con số báo cáo cuối cùng sẽ lạc quan giả tạo. Ta dùng dự đoán *out-of-fold*: mỗi mẫu huấn luyện được chấm điểm bởi mô hình chưa từng nhìn thấy nó.

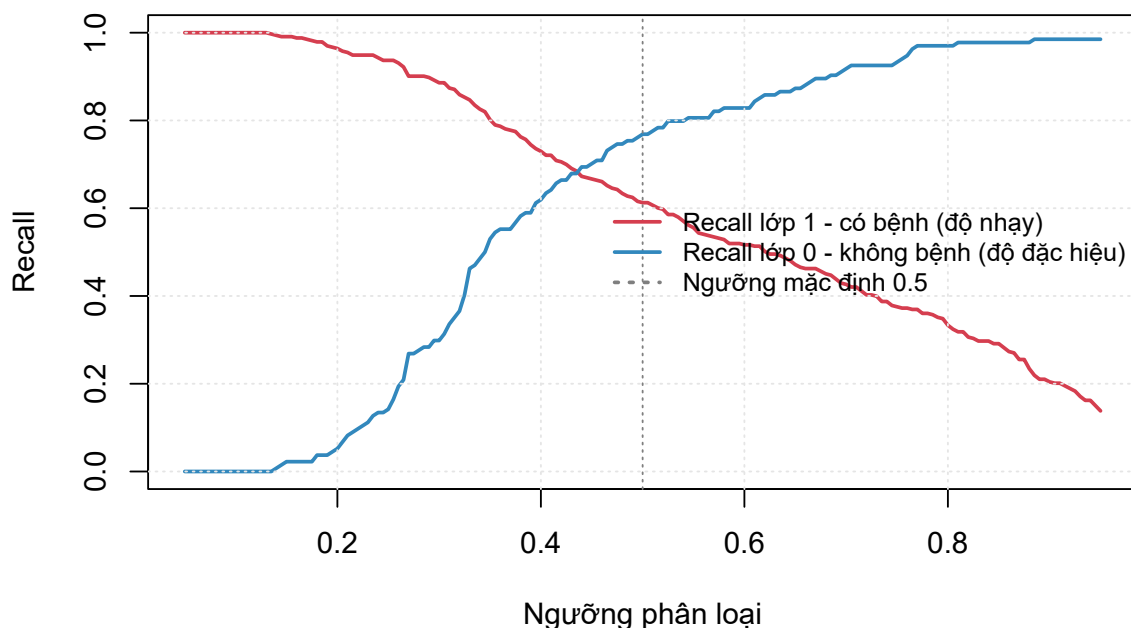
```
oof_proba <- numeric(length(y_train))
for (i in seq_len(5)) {
  te <- which(folds_inner == i); tr <- which(folds_inner != i)
  fit_o <- fit_pipeline(X_train[tr, , drop = FALSE], y_train[tr],
    lambda = best$lambda, smote_k = best$smote_k)
  oof_proba[te] <- pipeline_proba(fit_o, X_train[te, , drop = FALSE])
}

grid_thr <- seq(0.05, 0.95, by = 0.005)
rec1 <- rec0 <- numeric(length(grid_thr))
for (i in seq_along(grid_thr)) {
  m <- compute_metrics(y_train, as.integer(oof_proba >= grid_thr[i]))
  rec1[i] <- m["recall1"]; rec0[i] <- m["recall0"]
}
```

```
plot(grid_thr, rec1, type = "l", col = "#d53e4f", lwd = 2, ylim = c(0, 1),
  xlab = "Ngưỡng phân loại", ylab = "Recall",
  main = "Recall hai lớp theo ngưỡng (out-of-fold trên tập train)")
lines(grid_thr, rec0, col = "#3288bd", lwd = 2)
abline(v = 0.5, col = "gray50", lty = 3)
grid(col = "gray90")
legend("right", bty = "n", cex = 0.85, lwd = 2,
  legend = c("Recall lớp 1 - có bệnh (độ nhạy)",
    "Recall lớp 0 - không bệnh (độ đặc hiệu)",
    "Ngưỡng mặc định 0.5"),
  col = c("#d53e4f", "#3288bd", "gray50"), lty = c(1, 1, 3))
```

Hình 6 thể hiện đúng bản chất đánh đổi: hai đường đi ngược chiều nhau, mọi điểm recall lớp 1 tăng thêm đều phải trả bằng recall lớp 0 giảm đi. Bảng tra dưới đây lượng hoá cái giá đó.

Recall hai lớp theo ngưỡng (out-of-fold trên tập train)



Hình 6: Đánh đổi giữa độ nhạy và độ đặc hiệu khi thay đổi ngưỡng phân loại

```
lookup <- data.frame()
for (target in c(0.70, 0.75, 0.80, 0.85, 0.90, 0.95)) {
  ok <- which(rec1 >= target)
  if (length(ok) == 0) next
  i <- ok[which.max(grid_thr[ok])] # ngưỡng CAO NHẤT vẫn đạt mục tiêu
  lookup <- rbind(lookup, data.frame(
    recall_muc_tieu = target, nguong = grid_thr[i],
    recall_lop1 = rec1[i], recall_lop0 = rec0[i]
  ))
}
print(round(lookup, 3), row.names = FALSE)

## recall_muc_tieu nguong recall_lop1 recall_lop0
##      0.70  0.420      0.706      0.664
##      0.75  0.385      0.757      0.590
##      0.80  0.350      0.802      0.530
##      0.85  0.325      0.853      0.403
##      0.90  0.285      0.901      0.284
##      0.95  0.210      0.955      0.082

TARGET_RECALL <- 0.70
ok <- which(rec1 >= TARGET_RECALL)
threshold <- grid_thr[ok[which.max(grid_thr[ok])]]

cat(sprintf("\nNgưỡng mặc định : 0.500\n"))

##
## Ngưỡng mặc định : 0.500

cat(sprintf("Ngưỡng đã chọn : %.3f (mục tiêu recall >= %.2f)\n",
  threshold, TARGET_RECALL))

## Ngưỡng đã chọn : 0.420 (mục tiêu recall >= 0.70)
```

Bảng tra cho thấy cái giá tăng rất nhanh. Để đạt recall 0,70 ta chỉ mất một ít độ đặc hiệu (còn 0,664); nhưng để đạt 0,90 thì độ đặc hiệu tụt xuống 0,284, và ở mức 0,95 chỉ còn 0,082 — lúc đó mô hình gần như báo “có bệnh” cho tất cả mọi người và mất hết giá trị sàng lọc. Mức 0,70 được chọn vì nó nằm ở đoạn đánh đổi còn cân bằng, tương ứng ngưỡng 0,420.

10 Đánh giá cuối cùng trên tập kiểm tra

Đây là lần **đầu tiên** tập kiểm tra được dùng tới kể từ khi niêm phong. Mô hình được khớp lại trên toàn bộ tập huấn luyện với siêu tham số đã chọn, rồi áp ngưỡng 0,420.

```
fit_best <- fit_pipeline(X_train, y_train,
                        lambda = best$lambda, smote_k = best$smote_k)
proba_final <- pipeline_proba(fit_best, X_test)
pred_final <- as.integer(proba_final >= threshold)
pred_05 <- as.integer(proba_final >= 0.5)

m_final <- print_confusion(y_test, pred_final,
                          sprintf("CONFUSION MATRIX (ngưỡng %.3f)",
                                  threshold))

##
## CONFUSION MATRIX (ngưỡng 0.420)
##          Dự đoán
## Thực tế  Không bệnh  Có bệnh
## Không bệnh      27      6
## Có bệnh         25     58
## TN = 27   không bệnh, đoán đúng
## FP = 6    không bệnh nhưng báo có bệnh
## FN = 25   CÓ BỆNH nhưng bỏ sót
## TP = 58   có bệnh, bắt được

cat("\nCác chỉ số trên tập test:\n")

##
## Các chỉ số trên tập test:

print(round(m_final, 3))

## accuracy recall1 precision1 recall0 precision0 macro_f1
##      0.733      0.699      0.906      0.818      0.519      0.712
```

```
cmp <- rbind(
  `0.500 (mặc định)` = compute_metrics(y_test, pred_05),
  `đã chỉnh`        = compute_metrics(y_test, pred_final)
)
print(round(cmp, 3))

##          accuracy recall1 precision1 recall0 precision0
## 0.500 (mặc định)  0.681  0.614      0.911  0.848      0.467
## đã chỉnh         0.733  0.699      0.906  0.818      0.519
##          macro_f1
## 0.500 (mặc định)  0.668
## đã chỉnh         0.712
```

Việc hạ ngưỡng từ 0,500 xuống 0,420 đẩy recall lớp có bệnh từ 0,614 lên 0,699 — bắt thêm được 7 ca bệnh — với cái giá là độ đặc hiệu giảm nhẹ từ 0,848 xuống 0,818 (thêm 1 báo động nhầm). Điều đáng chú ý là accuracy và macro F1 *cũng* tăng, chứ không giảm như thường thấy khi ưu tiên một lớp. Nguyên nhân là ngưỡng 0,5 vốn không tối ưu trên dữ liệu mất cân bằng này ngay cả theo tiêu chí tổng hợp; mức 0,420 tình cờ tốt hơn ở cả hai phương diện.

10.1 Khoảng tin cậy bootstrap cho toàn bộ chỉ số

```
CONF2 <- 0.95
B2 <- 2000
boot_mat <- matrix(NA, nrow = B2, ncol = length(m_final),
  dimnames = list(NULL, names(m_final)))

for (b in seq_len(B2)) {
  idx <- sample.int(n_test, n_test, replace = TRUE)
  if (length(unique(y_test[idx])) < 2) next # mẫu suy biến, bỏ qua
  boot_mat[b, ] <- compute_metrics(y_test[idx], pred_final[idx])
}
boot_mat <- boot_mat[complete.cases(boot_mat), , drop = FALSE]

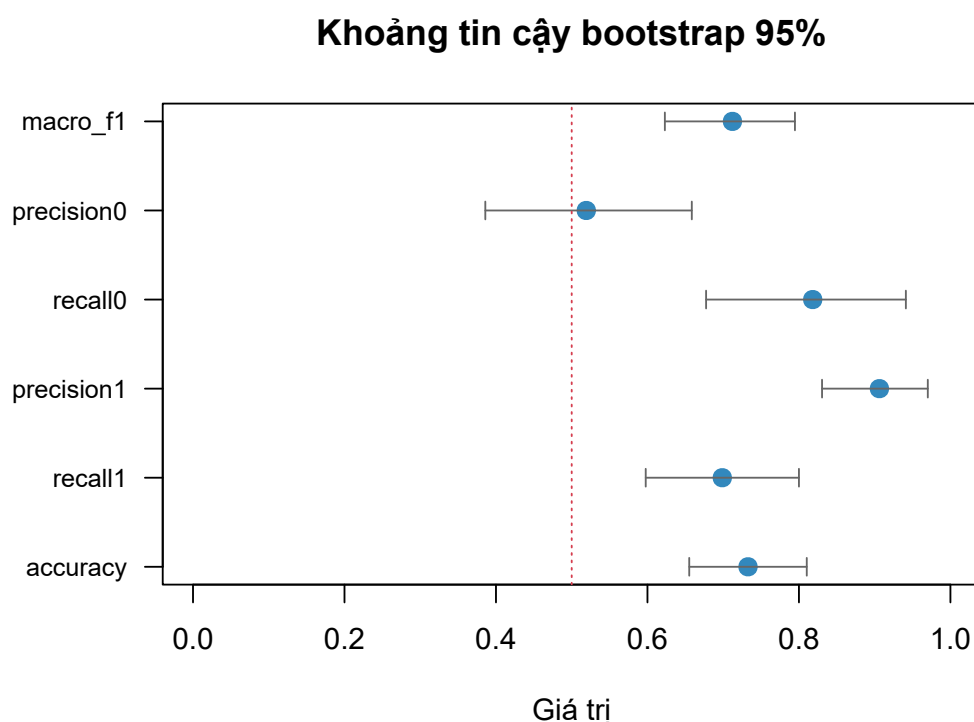
tail2 <- (1 - CONF2) / 2
ci_tbl <- data.frame(
  diem_uoc_luong = m_final,
  ci_duoi = apply(boot_mat, 2, quantile, probs = tail2),
  ci_tren = apply(boot_mat, 2, quantile, probs = 1 - tail2),
  sai_so_chuan = apply(boot_mat, 2, sd)
)
print(round(ci_tbl, 4))

##           diem_uoc_luong ci_duoi ci_tren sai_so_chuan
## accuracy           0.7328  0.6552  0.8103         0.0413
## recall1            0.6988  0.5977  0.8000         0.0512
## precision1         0.9062  0.8305  0.9701         0.0368
## recall0            0.8182  0.6774  0.9412         0.0676
## precision0         0.5192  0.3859  0.6586         0.0693
## macro_f1           0.7122  0.6230  0.7947         0.0437
```

```
par(mar = c(5, 9, 4, 2))
n_m <- nrow(ci_tbl)
plot(ci_tbl$diem_uoc_luong, seq_len(n_m), pch = 19, col = "#3288bd",
  cex = 1.3, xlim = c(0, 1), yaxt = "n", ylab = "", xlab = "Giá trị",
  main = sprintf("Khoảng tin cậy bootstrap %.0f%%", CONF2 * 100))
arrows(ci_tbl$ci_duoi, seq_len(n_m), ci_tbl$ci_tren, seq_len(n_m),
  length = 0.05, angle = 90, code = 3, col = "gray40")
axis(2, seq_len(n_m), rownames(ci_tbl), las = 2, cex.axis = 0.85)
abline(v = 0.5, col = "#d53e4f", lty = 3)
```

```
par(mar = c(5, 4, 4, 2) + 0.1)
```

Kết quả chính: **recall lớp có bệnh** = 0,699, **khoảng tin cậy 95%** là [0,598; 0,800]. Hình 7 cho thấy độ rộng khoảng tin cậy khác nhau đáng kể giữa các chỉ số, và sự khác biệt đó phản ánh trực tiếp cỡ mẫu dùng để tính từng chỉ số. precision1 có khoảng hẹp nhất ($\pm 0,07$) vì mẫu số của nó gồm 64 dự đoán dương tính; ngược lại precision0 có khoảng rộng nhất ([0,386; 0,659], sai



Hình 7: Khoảng tin cậy bootstrap 95% cho sáu chỉ số trên tập kiểm tra

số chuẩn 0,069) vì chỉ tính trên 52 dự đoán âm tính. Nói cách khác, ước lượng 0,519 cho `precision0` gần như không cho biết điều gì chắc chắn — một cảnh báo mà điểm ước lượng đơn lẻ hoàn toàn giấu đi.

11 Kết luận

Tiểu luận đã xây dựng một quy trình phân loại hoàn chỉnh trên dữ liệu ILPD và, quan trọng hơn, minh họa vai trò phân công rõ ràng giữa hai kỹ thuật resampling.

K-fold Cross-Validation trong vai trò chọn mô hình. Kiểm định chéo phân tầng 5 phần được dùng ở ba chỗ khác nhau: đánh giá tổng quát mô hình cơ sở ($0,643 \pm 0,031$ macro F1), dò 15 bộ siêu tham số qua 75 lần khớp mô hình, và sinh dự đoán out-of-fold để chọn ngưỡng phân loại. Điểm rút ra đáng giá nhất không phải bộ tham số thắng cuộc mà là phát hiện rằng 14 trên 15 bộ nằm trong phạm vi một sai số chuẩn của nhau — tức là lưới tham số này về cơ bản không phân biệt được các cấu hình, và việc chọn bộ có điểm cao nhất sẽ là chọn theo nhiều. Chính độ lệch chuẩn giữa các fold, thứ mà một lần chia train/test đơn lẻ không thể cung cấp, đã cho phép nhận ra điều đó.

Bootstrap trong vai trò định lượng độ tin cậy. Mô hình cuối đạt recall 0,699 trên tập kiểm tra, nhưng khoảng tin cậy 95% trải từ 0,598 tới 0,800. Bề rộng hơn 20 điểm phần trăm này là thông tin thiết yếu: nó nói rằng với cỡ mẫu

hiện có, ta không thể phân biệt được mô hình này với một mô hình có recall thật sự là 0,62 hay 0,78. Mọi so sánh với các nghiên cứu khác trên cùng bộ dữ liệu đều phải đọc qua lăng kính đó.

Về mô hình. Cấu hình cuối là hồi quy logistic ridge với $\lambda = 0,01$, SMOTE với $k = 3$ láng giềng, ngưỡng phân loại 0,420, trên 7 biến còn lại sau khi loại Direct_Bilirubin (đa cộng tuyến), Gender và Total_Protiens (không đạt FDR). Nhóm biến quan trọng nhất là các men gan và bilirubin, với kích thước ảnh hưởng rank-biserial khoảng 0,35–0,39.

11.1 Hạn chế

1. **Hiệu năng tuyệt đối còn thấp.** Recall 0,699 nghĩa là vẫn bỏ sót khoảng 30% ca bệnh (25 trên 83 bệnh nhân trong tập kiểm tra). Đặc biệt precision chỉ đạt 0,519: trong số những người mô hình kết luận là khoẻ mạnh, gần một nửa thực ra có bệnh. Đây chưa phải mức chấp nhận được cho một công cụ sàng lọc dùng trong thực tế.
2. **Khoảng tin cậy bootstrap chỉ tính một nguồn bất định.** Vì mô hình được cố định trong suốt vòng lặp và chỉ tập kiểm tra được rút lại, khoảng thu được phản ánh biến thiên do lấy mẫu bệnh nhân kiểm tra, chứ không tính tới biến thiên do thay đổi tập huấn luyện. Độ bất định thật sự do đó *rộng hơn* con số báo cáo. Khắc phục đúng cách là dùng bootstrap lồng — huấn luyện lại mô hình trong mỗi lần lặp — với chi phí tính toán cao hơn nhiều.
3. **Cỡ mẫu nhỏ.** 583 quan sát, trong đó tập kiểm tra chỉ 116, là nguyên nhân trực tiếp của các khoảng tin cậy rộng.
4. **Tính đại diện.** Dữ liệu thu thập ở một vùng địa lý duy nhất (đông bắc Andhra Pradesh, Ấn Độ) nên kết quả chưa chắc tổng quát hoá được sang quần thể khác.
5. **Đa cộng tuyến còn sót.** Albumin và Albumin_and_Globulin_Ratio vẫn liên quan chặt về định nghĩa; điều chuẩn ridge làm dịu vấn đề này nhưng không loại bỏ được nó, và hệ quả là các hệ số riêng lẻ của mô hình không nên được diễn giải như mức độ quan trọng của từng biến.

11.2 Hướng mở rộng

Ba hướng có triển vọng nhất, xếp theo mức độ ưu tiên. Thứ nhất, *bootstrap lồng* để khoảng tin cậy bao gồm cả biến thiên do huấn luyện. Thứ hai, *repeated stratified k-fold* — lặp toàn bộ quy trình CV với nhiều seed khác nhau — để

giảm phương sai của ước lượng và kiểm tra xem kết luận “14/15 bộ tham số ngang nhau” có ổn định không. Thứ ba, thử các họ mô hình phi tuyến (random forest, gradient boosting) vốn có thể nắm bắt tương tác giữa các men gan mà mô hình tuyến tính bỏ qua; khi đó chính khung K-fold đã dựng ở đây sẽ là công cụ so sánh khách quan giữa các họ mô hình.

12 Tài liệu tham khảo

1. Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.
2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
3. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26.
4. Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
6. Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
7. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
8. Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *Proceedings of IJCAI*, 1137–1143.
9. R Core Team (2026). *R: A Language and Environment for Statistical Computing* (version 4.6.1). R Foundation for Statistical Computing, Vienna, Austria.
10. Ramana, B., & Venkateswarlu, N. (2022). ILPD (Indian Liver Patient Dataset) [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5D02C>